



第07章 一维数组和C字符串

张仁斌@合肥工业大学软件学院

本章主要内容



7.1 数组的定义与简单使用



7.2 数组作为函数参数

7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回

7.5 搜索数组

7.6 排序数组

7.7 C字符串

7.8 作业与实践

引例



◆ 假设要读取100个整数，并对这些整数进行升序或降序排序，需要定义100个变量并重复编写大量相似代码读入、比较、交换数据？

- 读取100个整数并计算其平均值时，可以只定义3个变量分别用于接收输入、存储累加和、记录输入数据个数，但排序输入数据需同时保存输入的全部数据
- C/C++和大多数其它高级语言都提供了一种**数据结构**——**数组**（array）用于存储具有**相同数据类型**的数据集合（集合中每个元素的大小固定）
- 可以用数组接收输入数据并排序、输出排序结果

```
int numbers[100];
for(int i = 0; i < 100; i++)
{
    cout << "Enter a new number: ";
    cin >> numbers[i];
}

// 排序
// 输出排序结果
```

本章仅介绍一维数组，将在下一章介绍二维和多维数组



定义数组变量

◆ 定义数组变量须指明数组元素数据类型和**数组长度**，语法为：

一维数组：元素数据类型 数组名[常量表达式]

二维数组：元素数据类型 数组名[常量表达式][常量表达式]

三维数组：.....

.....

```
double myList[10];  
const int SIZE = 5;  
int list1[SIZE], list2[SIZE];
```

◆ 元素数据类型可以是任何数据类型，所有的数组元素（数组成员）都是**同样数据类型**

◆ 常量表达式的值必须是大于0的整数

- 在 **C89** 中，必须使用**常量表达式**指明数组长度（数组**元素个数**），也就是说，指明数组长度的表达式中不能包含变量，不管该变量是否已初始化

例如：int count = 5; double myList[count]; // 非法

- 在 **C99** 中，可以使用**变量**指明数组长度，但**C11**取消了此特性

- 各种编译器都能很好地支持 C89 标准，但对 C99 的支持却不同

本课程依据C89



◆ 编译器为定义的数组分配**连续**的内存空间存储数组的每个元素

float myList[5];

数组长度



元素索引值

- 数组名代表数组在内存中的起始地址
- 数组元素个数是固定的，在程序执行中固定不变
- 数组元素在内存中按顺序连续存储，可以使用数组下标访问数组元素

- 下标index的取值范围是： **$0 \leq \text{index} \leq \text{数组长度}-1$**
- myList[0]是数组的第1个元素，myList[4]是数组的最后1个元素
- 使用数组下标访问时，数组中的**每个元素都可以当作变量来使用**

```
myList[2] = myList[0] + myList[1]; // 假设数组已初始化
myList[0]++;
myList[3] = 10;
cin >> myList[4];
```

下标可以是整型常量、整型变量或整型表达式，编号从0开始，且不能越界



初始化数组

◆ 可以使用数组下标给数组元素赋值、初始化数组

■ 数组元素赋值语法: `arrayName[index] = value;`

◆ 定义并初始化数组

```
float myList[5];  
myList[0] = 1.2;  
myList[1] = 2.3;  
myList[2] = 3.4;  
myList[3] = 4.5;  
myList[4] = 5.6;
```

等价于

```
float myList[5] = {1.2, 2.3, 3.4, 4.5, 5.6};
```

等价于

```
float myList[ ] = {1.2, 2.3, 3.4, 4.5, 5.6};
```

根据右边花括号中数据个数确定数组长度

```
float myList[5];  
myList[5] = {1.2, 2.3, 3.4, 4.5, 5.6};
```

◆ C++ 允许初始化数组时只给定部分元素

```
float myList[5] = {1.2, 2.3};
```

等价于

```
float myList[5] = {1.2, 2.3, 0, 0, 0};
```

由编译器完成
“补零”

■ 花括号中数值个数不可以超过数组长度

```
float myList[5] = {1, 2, 3, 4, 5, 6, 7, 8};
```



- ◆ 如果想使一个数组中**全部**元素的值都为0，则可以：

```
float myList[5] = {0};
```

- ◆ 只有在定义数组时，可以用{ }给数组整体赋值，其他情况，只能逐个元素赋值
- ◆ 如果只定义了数组，未进行初始化，则其元素的值都是“垃圾”（不确定的内容，未初始化的普通变量的值也是如此）



对数组的常用简单处理

◆ 常利用for循环处理数组元素，因为：

- 数组中全部元素都是相同类型的，使用循环能反复以同样方式一致地处理所有元素
- 数组长度（数组元素个数）已知，用for循环比较直观、自然

◆ 基于以下定义的数组，介绍对数组的常用简单处理

- `const int ARRAY_SIZE = 10;`
- `double myList[ARRAY_SIZE];`

◆ （1）利用输入值初始化数组

```
cout << "Enter " << ARRAY_SIZE << " values: ";  
for(int i = 0; i < ARRAY_SIZE; i++)  
    cin >> myList[i];
```




◆ (2) 利用随机数初始化数组（假设限制随机数为0~99）

```
srand(time(0));  
for(int i = 0; i < ARRAY_SIZE; i++)  
    myList[i] = rand() % 100;
```

◆ (3) 输出数组

```
for(int i = 0; i < ARRAY_SIZE; i++)  
    cout << myList[i] << " ";  
cout << endl;
```

◆ (4) 复制数组（C/C++不允许采用list = myList复制数组myList）

➤ 只能在两个数组之间逐一复制全部元素

```
for(int i = 0; i < ARRAY_SIZE; i++)  
    list[i] = myList[i];
```



◆ (5) 求数组全部元素的和

```
double sum = 0;
for(int i = 0; i < ARRAY_SIZE; i++)
    sum += myList[i];
```

◆ (6) 求数组中最大元素

```
double max = myList[0];
for(int i = 1; i < ARRAY_SIZE; i++)
    if(myList[i] > max) max = myList[i];
```

◆ (7) 求数组中最大元素的~~最小~~下标

```
double max = myList[0];
int indexOfMax = 0
for(int i = 1; i < ARRAY_SIZE; i++) {
    if(myList[i] > max) {
        max = myList[i];
        indexOfMax = i;
    }
}
```

若将其改为 **myList[i] >= max**, 结果会如何?
假设myList为{1, 5, 3, 4, 5}



◆ (8) 随机重排 (随机交换数组中的元素)

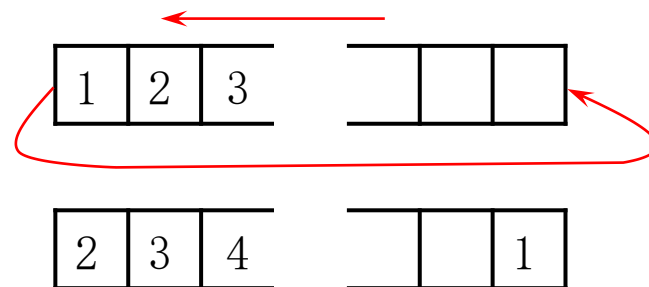
```
 srand(time(0));  
 for(int i = ARRAY_SIZE - 1; i > 0; i--)  
 {  
     int j = rand( ) % (i + 1);  
     double tmp = myList[i];  
     myList[i] = myList[j];  
     myList[j] = tmp;  
 }
```

如此初始化i，是为了便于产生指定范围的下标

◆ (9) 移动元素 (向左或向右移动元素)，例如将元素左移一个位置 (全部元素都依次左移一个位置)，然后将原来的第一个元素填写到最右边 (最后) 一个位置

```
 double tmp = myList[0];  
 for(int i = 1; i < ARRAY_SIZE; i ++)  
     myList[i - 1] = myList[i];  
 myList[ARRAY_SIZE - 1] = tmp;
```

如何实现右移?





◆ (10) 简化代码

■ 数组可以在某些任务中简化代码。例如，利用月份数字快速获取月份英文名：

```
string months[ ] = {"January", "February", ....., "December"};  
cout << Enter a month number(1 to 12): ";  
int monthNumber;  
cin >> monthNumber;  
cout << "The month is " << months[monthNumber - 1] << endl;
```

此处假设输入数据有效；
C/C++不自动检查是否越界

■ 若不使用数组，则需使用冗长的多分支if-else语句或switch语句来确定月份名：

```
if(monthNumber == 1)  
    cout << "The month is January" << endl;  
else if(monthNumber == 2)  
    cout << "The month is February" << endl;  
...  
else  
    cout << "The month is December" << endl;
```

常见错误与输出示例



下列语句是有效的数组定义吗？

```
double d[30];  
char[30]✗r;  
int i[ ] = (3, 4, 3, 2)✗;  
float f[ ] = {2.3, 4.5, 6.6};
```

下列代码的输出是什么

```
int list[ ] = {1, 2, 3, 4, 5, 6};  
for(int i = 1; i < 6; i++)  
    list[i] = list[i-1];  
for(int i = 0; i < 6; i++)  
    cout << list[i] << " ";
```

识别并修改下列代码中的错误

```
int main( )  
{  
    double[100] ✗r;  
  
    for(int i = 0; i < 100; i++)✗;  
        r(i)✗ = rand( ) % 100;  
}
```

示例：签到（教材中彩票号码问题简化）



- ◆ 假设小组有50位同学，代号分别为1~50，输入代号进行签到，输入0或无效代号时表示签到结束，统计是否有同学未签到

```
#include <iostream>
using namespace std;
int main() {
    bool isPresent[5];
    int number;

    // Initialize the array
    for (int i = 0; i < 5; i++)
        isPresent[i] = false;

    cout << "Enter your number: ";
    cin >> number;
    while (number > 0 && number <= 5) // 签到
    {
        isPresent[number - 1] = true;
        cin >> number;
    }
```

最好是用命名常量替代字面常量，以保持一致性

```
// Check if all is present
bool allPresent = true;
for (int i = 0; i < 5; i++)
    if (!isPresent[i])
    {
        allPresent = false; // 发现有人未到
        break;
    }

// Display result
if (allPresent)
    cout << "都到了" << endl;
else
    cout << "有同学未到" << endl;

return 0;
}
```

D:\Edu-CPP\chap07\ex07-01.exe

```
Enter your number: 1
5
4
2
9
有同学未到
```

示例：数据统计分析



◆ 读入100个数，计算其平均值，并统计超过平均值的数的个数

```
#include <iostream>
using namespace std;
int main() {
    const int NUMBER_OF_ELEMENTS = 3; // 便于快速变更数据个数，并保持代码中数据的一致性
    double numbers[NUMBER_OF_ELEMENTS];
    double sum = 0;
    for (int i = 0; i < NUMBER_OF_ELEMENTS; i++) {
        cout << "Enter a new number: ";
        cin >> numbers[i];
        sum += numbers[i];
    }
    double average = sum / NUMBER_OF_ELEMENTS;
    int count = 0; // The number of elements above average
    for (int i = 0; i < NUMBER_OF_ELEMENTS; i++)
        if (numbers[i] > average) count++;
    cout << "Average is " << average << endl;
    cout << "Number of elements above the average " << count << endl;
    return 0; }
```

D:\Edu-CPP\chap07\ex07-02.exe

```
Enter a new number: 1
Enter a new number: 2
Enter a new number: 3
Average is 2
Number of elements above the average 1
```

示例：斐波纳契(Fibonacci)数列



- ◆ 用数组处理求斐波纳契(Fibonacci)数列：1, 1, 2, 3, 5, 8, ...的前20个数。数学表示： $f(0)=f(1)=1$, $f(n)=f(n-2)+f(n-1)$

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    int f[20] = {1, 1};
    for(int i = 2; i < 20; i++)
        f[i] = f[i - 2] + f[i - 1];
    for(int i = 0; i < 20; i++)
    {
        if(i % 5 == 0)
            cout << endl;
        cout << setw(6) << f[i];
    }
    return 0;
}
```

为什么多出这个空行？如何去除？

D:\Edu-CPP\chap07\ex07-03.exe

1	1	2	3	5
8	13	21	34	55
89	144	233	377	610
987	1597	2584	4181	6765

D:\Edu-CPP\chap07\ex07-03.exe

1	1	2	3	5
8	13	21	34	55
89	144	233	377	610
987	1597	2584	4181	6765

本章主要内容



7.1 数组的定义与简单使用

7.2 数组作为函数参数



7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回

7.5 搜索数组

7.6 排序数组

7.7 C字符串

7.8 作业与实践

在函数中输出参数数组的内容



- ◆ 函数的参数可以是数组，当一个实参数组传递给一个函数时，该数组的起始地址被传递给函数中的形参数组（**传地址**，简称**传址**），**实参和形参使用同一个数组（共享数组内存空间）**

```
#include <iostream>
using namespace std;
```

```
void printArray(int list[ ], int arraySize); // 函数声明（原型），可简化为：void printArray(int [ ], int);
```

```
int main() {
    int numbers[5] = {1, 4, 3, 6, 8};
    printArray(numbers, 5);
    return 0;
}
```

数组名代表数组在内存中的起始地址

numbers



```
void printArray(int list[ ], int arraySize) {
    for (int i = 0; i < arraySize; i++)
        cout << list[i] << " ";
}
```

任意整型数组，故需传入数组长度方可遍历数组

```
D:\Edu-CPP\chap07\ex07-04.exe
```

```
1 4 3 6 8
```

在函数中改变数组的元素值



- ◆ 数组参数的传递是传地址，形参和实参都是指向同一数组的指针，可以在函数中更新该数组

```
#include <iostream>
using namespace std;

void m(int, int [ ]);

int main() {
    int x = 1; // x represents an int value
    int y[10]; // y represents an array of int values
    y[0] = 1; // Initialize y[0]

    m(x, y); // Invoke m with arguments x and y

    cout << "x is " << x << endl;
    cout << "y[0] is " << y[0] << endl;

    return 0;
}
```

```
void m(int number, int numbers[ ])
{
    number = 1001; // Assign a new value to number
    numbers[0] = 5555; // Assign a new value to numbers[0]
}
```



函数m被调用之后，x的值仍为1，但y[0]的值变为5555，因为x是传值，数组y是传址

```
> D:\Edu-CPP\chap07\ex07-05.exe
```

```
x is 1
y[0] is 5555
```

本章主要内容



- 7.1 数组的定义与简单使用
- 7.2 数组作为函数参数
- 7.3 防止函数修改传递参数的数组
- 7.4 数组作为函数值返回
- 7.5 搜索数组
- 7.6 排序数组
- 7.7 C字符串
- 7.8 作业与实践



防止函数修改数组参数



- ◆ 传递数组参数其实是传递数组在内存中的首地址，数组元素没有被复制，提高了参数传递的效率，但有时只是让函数访问利用数据，而不希望函数修改数组，此时，可以利用**关键字const**告诉编译器此数组不能被修改，若代码尝试修改数组，则产生**编译错误**

```
#include <iostream>
using namespace std;

void p(const int list[ ], int arraySize)
{
    list[0] = 100; // 编译错误
}

int main( ) {
    int numbers[5] = {1, 2, 3, 4, 5};
    p(numbers, 5);
    return 0;
}
```

```
1  #include <iostream>
2  using namespace std;
3
4  void p(const int list[ ], int arraySize)
5  {
6      list[0] = 100; // 编译错误
7  }
8
9  int main( ) {
10     int numbers[5] = {1, 2, 3, 4, 5};
11     p(numbers, 5);
12     return 0;
13 }
```

es Compile Log Debug Find Results Close

Message

In function 'void p(const int*, int)':

[Error] assignment of read-only location '* list'



◆函数之间传递const变量时，对应参数必须都是const类型，否则产生编译错误

```
#include <iostream>
using namespace std;
```

```
void f2(int list[ ], int arraySize)
{
    // do somethin
}
```

须定义为**const int list[]**，避免
const int *转int *的编译时错误，
编译错误也说明数组名是指针

```
void f1(const int list[ ], int arraySize)
{
    f2(list, arraySize); // 编译错误
}
```

```
1  #include <iostream>
2  using namespace std;
3
4  void f2(int list[ ], int arraySize)
5  {
6      // do somethin
7  }
8
9  void f1(const int list[ ], int arraySize)
10 {
11     f2(list, arraySize); // 编译错误
12 }
13 void f2 (int list[ ], int arraySize)
```

ces Compile Log Debug Find Results Close

Message

In function 'void f1(const int*, int)':

[Error] invalid conversion from 'const int*' to 'int*' [-fpermissive]

[Note] initializing argument 1 of 'void f2(int*, int)'

本章主要内容



7.1 数组的定义与简单使用

7.2 数组作为函数参数

7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回



7.5 搜索数组

7.6 排序数组

7.7 C字符串

7.8 作业与实践



◆函数返回值数据类型可以是基本数据类型:

int sum(const int list[], int arraySize) // 函数返回值为参数指定数组元素之和

■但不能是数组

int [] reverse(const int list[], int arraySize) // 期望返回参数指定数组的逆序数组

■但可以向函数传递两个数组，绕过此限制

void revers(const int list[], **int newList[]**, int arraySieve) // 利用newList返回逆序

获取函数处理结果，可以：

【1】利用函数返回值；

【2】利用传引用或传地址的参数

示例：获取数组的逆序数组



```
#include <iostream>
using namespace std;

void reverse(const int list[], int newList[], int size)
{
    for (int i = 0, j = size - 1; i < size; i++, j--)
        newList[j] = list[i];
}

void printArray(const int list[], int size)
{
    for (int i = 0; i < size; i++)
        cout << list[i] << " ";
}
```

D:\Edu-CPP\chap07\ex07-08.exe

```
The original array: 1 2 3 4 5 6
The reversed array: 6 5 4 3 2 1
```

```
int main()
{
    const int SIZE = 6;
    int list[] = {1, 2, 3, 4, 5, 6};
    int newList[SIZE];

    reverse(list, newList, SIZE);

    cout << "The original array: ";
    printArray(list, SIZE);
    cout << endl;

    cout << "The reversed array: ";
    printArray(newList, SIZE);
    cout << endl;

    return 0;
}
```



◆将原数组逆序

```
#include <iostream>
using namespace std;

void reverse(int list[], int size)
{
    for (int i = 0; i < size / 2; i++)
    {
        int tmp = list[i];
        list[i] = list[size - 1 - i];
        list[size - 1 - i] = tmp;
    }
}
```

手工验证条件表达式的正确性（分别取4、5比较小的奇数和偶数）

```
int main()
{
    int size = 5;
    int list[] = {1, 2, 3, 4, 5};

    reverse(list, size);
    for (int i = 0; i < size; i++)
        cout << list[i] << " ";

    return 0;
}
```

D:\Edu-CPP\chap07\ex07-08B.exe

5 4 3 2 1



◆ 假设下面代码用于反转字符串中的字符，但实际输出结果未反转，错在哪里？

```
string s = "ABCD";  
for(int i = 0, j = s.size() - 1; i < s.size(); i++, j--)  
{  
    char tmp = s[i];  
    s[i] = s[j];  
    s[j] = tmp;  
}  
cout << "The reversed string is " << s << endl;
```

D:\Edu-CPP\chap07\ex07-09.exe

The reversed string is ABCD

D:\Edu-CPP\chap07\ex07-09.exe

The reversed string is DCBA

示例：统计每个字母出现的次数



◆要求：随机生成100个小写字母，统计每个字符出现的次数

◆分析

■随机生成小写字母的语句是：`static_cast<char>('a' + rand() % ('z' - 'a' + 1))`

■定义一个长度为100的字符数组存储随机生成的小写字母

■定义一个**长度为26**的整数数组，用于记录26个小写字母每个字母出现的次数

➤ **按'a'到'z'顺序记录**，即第1个元素记录'a'出现的次数，第2个元素记录'b'出现的次数，.....

```
#include <iostream>
#include <ctime>
#include <cstdlib>
using namespace std;

// 全局变量和函数声明
const int NUMBER_OF_LETTERS = 26;
const int NUMBER_OF_RANDOM_LETTERS = 100;
void createArray(char []);
void displayArray(const char []);
void countLetters(const char [], int []);
void displayCounts(const int []);
```

```
int main() {
    // Declare and create an array
    char chars[NUMBER_OF_RANDOM_LETTERS];

    // Initialize the array with random lowercase letters
    createArray(chars);

    // Display the array
    cout << "The lowercase letters are: " << endl;
    displayArray(chars);

    // Count the occurrences of each letter
    int counts[NUMBER_OF_LETTERS];

    // Count the occurrences of each letter
    countLetters(chars, counts);

    // Display counts
    cout << "\nThe occurrences of each letter are: " << endl;
    displayCounts(counts);

    return 0;
}
```

```
// Create an array of characters
void createArray(char chars[])
{
    // Create lowercase letters randomly and assign them to the array
    srand(time(0));
    for (int i = 0; i < NUMBER_OF_RANDOM_LETTERS; i++)
        chars[i] = static_cast<char>('a' + rand() % ('z' - 'a' + 1));
}

// Display the array of characters
void displayArray(const char chars[])
{
    // Display the characters in the array 20 on each line
    for (int i = 0; i < NUMBER_OF_RANDOM_LETTERS; i++)
    {
        if ((i + 1) % 20 == 0)
            cout << chars[i] << " " << endl;
        else
            cout << chars[i] << " ";
    }
}
```

```
// Count the occurrences of each letter
void countLetters(const char chars[], int counts[])
{
    // Initialize the array
    for (int i = 0; i < NUMBER_OF_LETTERS; i++)
        counts[i] = 0;

    // For each lowercase letter in the array, count it
    for (int i = 0; i < NUMBER_OF_RANDOM_LETTERS; i++)
        counts[chars[i] - 'a']++;
}

// Display counts
void displayCounts(const int counts[])
{
    for (int i = 0; i < NUMBER_OF_LETTERS; i++)
    {
        if ((i + 1) % 10 == 0)
            cout << counts[i] << " " << static_cast<char>(i + 'a') << endl;
        else
            cout << counts[i] << " " << static_cast<char>(i + 'a') << " ";
    }
}
```

> D:\Edu-CPP\chap07\ex07-10.exe

```
The lowercase letters are:
g e p g l o r p k m t a c f j m c q m h
b t h y k y e i o w h b o h u y v z f y
k d c t l n n z k v s r e g w k d u w u
e h c g h b m l q l f h h m g n w s p z
e q b b b w k w d b q j e j s b r e i z

The occurrences of each letter are:
1 a 8 b 4 c 3 d 7 e 3 f 5 g 8 h 2 i 3 j
6 k 4 l 5 m 3 n 3 o 3 p 4 q 3 r 3 s 3 t
3 u 2 v 6 w 0 x 4 y 4 z
```

利用counts[0]记录'a'次数、counts[1]记录'b'次数、.....，巧妙简化判断与统计，否则：
 if(chars[i] == 'a')
 counts[0]++;
 else if(chars[i] == 'b')
 counts[1]++;



示例：合并有序数组

◆合并两个有序的数组，合并后仍然有序，假设是升序

■例如将 [3, 4, 5, 8, 12] 与 [1, 2, 7, 8, 10, 34, 48] 合并为 [1, 2, 3, 4, 5, 7, 8, 8, 10, 12, 34, 48]

```
#include <iostream>
using namespace std;
```

最好是将两数据源数组定义为：

const int list1[], **const** int list2[]

```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

给result[k]赋值后k增1；提取list1[i]或list2[j]的值之后i增1或j增1

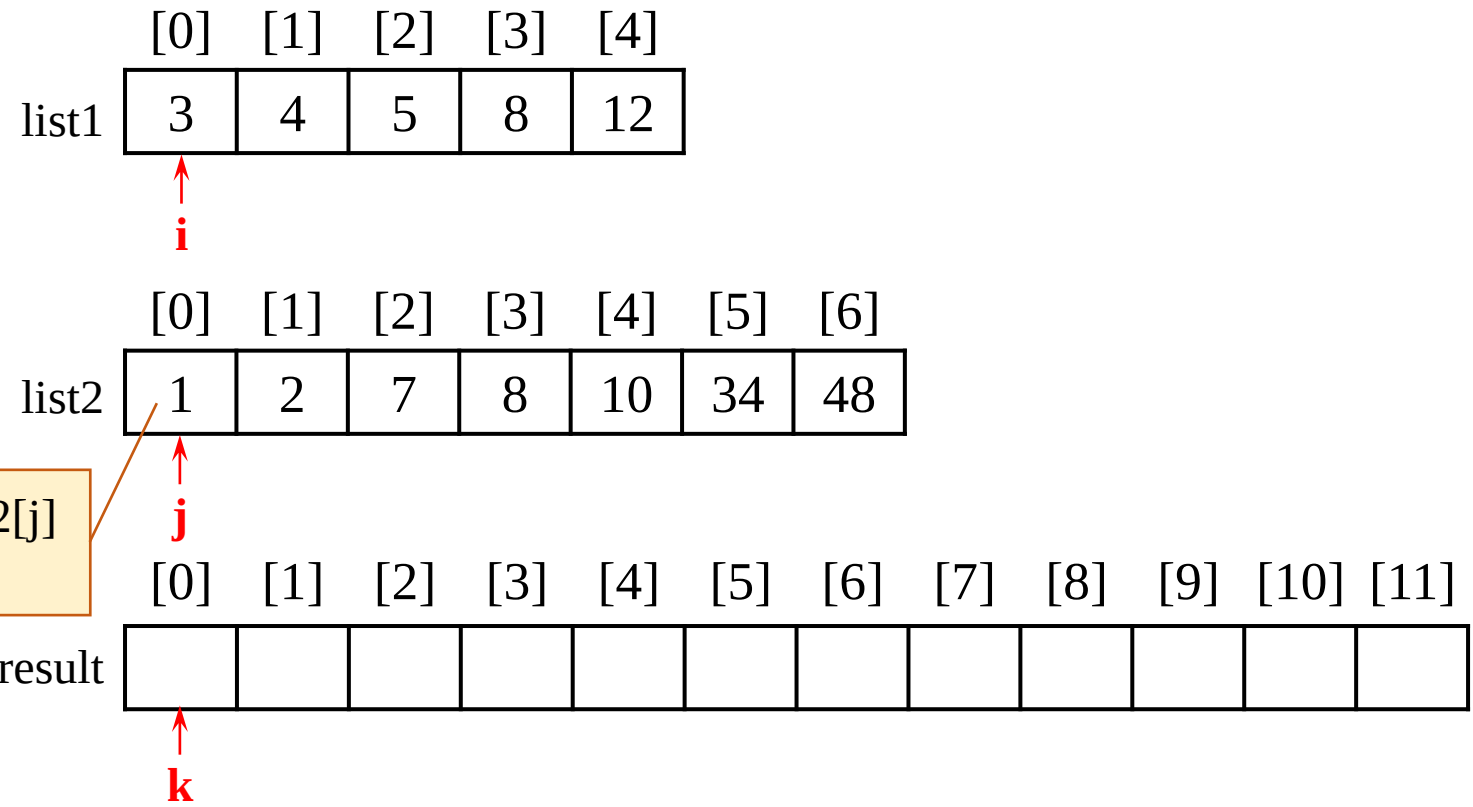
```
int main() {
    int list1[5] = {3, 4, 5, 8, 12};
    int list2[7] = {1, 2, 7, 8, 10, 34, 48};
    int result[12];

    merge(list1, list2, 5, 7, result);
    for(int i=0; i<12; i++)
        cout << result[i] << " ";
    return 0;
}
```

D:\Edu-CPP\chap07\ex07-11.exe

1 2 3 4 5 7 8 8 10 12 34 48


```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```



list1[i] <= list2[j]为假，提取list2[j]
赋给result[k]，k增1，j增1

```

void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}

```

	[0]	[1]	[2]	[3]	[4]
list1	3	4	5	8	12

↑
i

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
list2	1	2	7	8	10	34	48

↑
j

list1[i] <= list2[j]为假，提取list2[j]
赋给result[k]，k增1，j增1

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]
result	1											

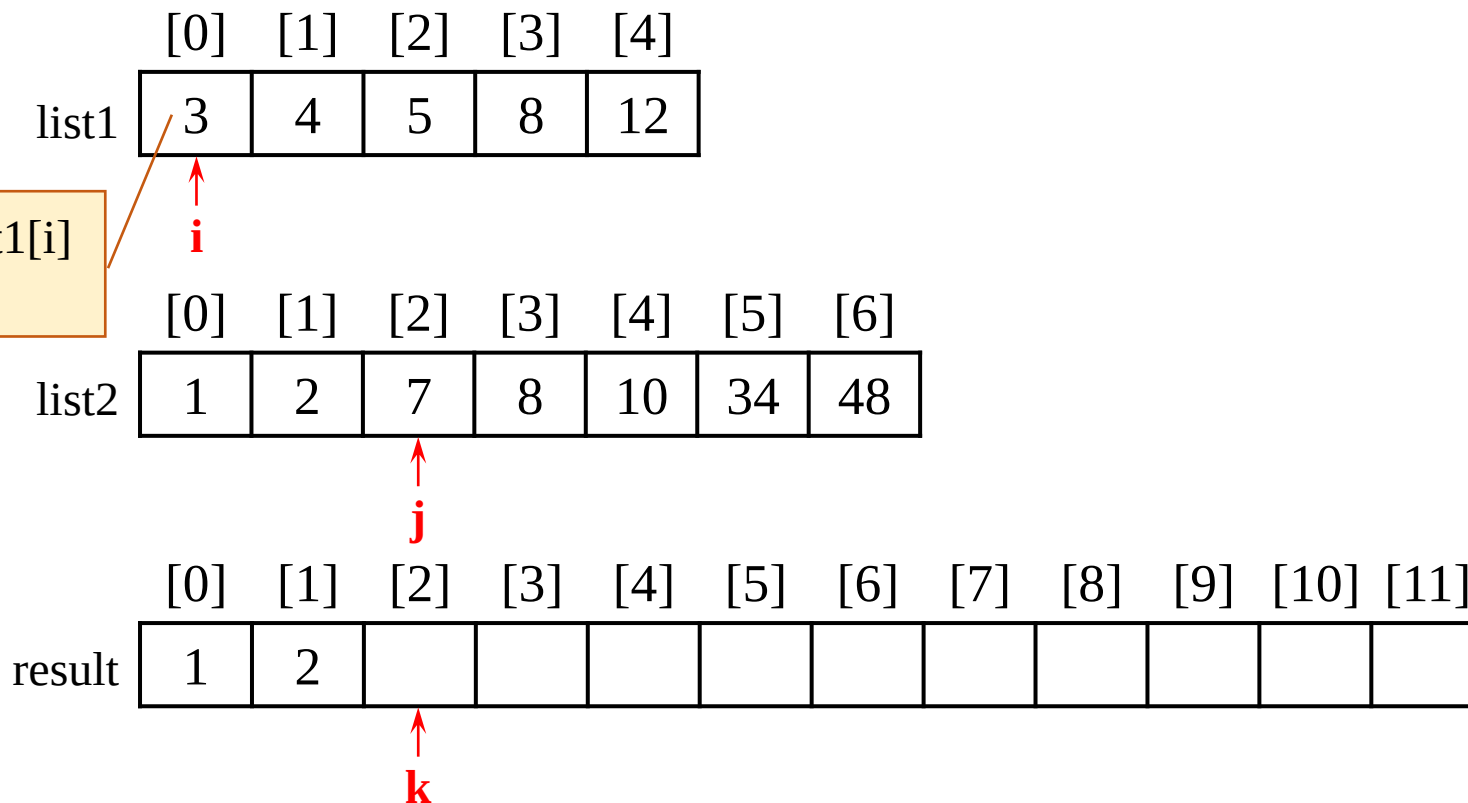
↑
k

```

void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}

```

list1[i] <= list2[j]为真，提取list1[i]
赋给result[k]，k增1，i增1

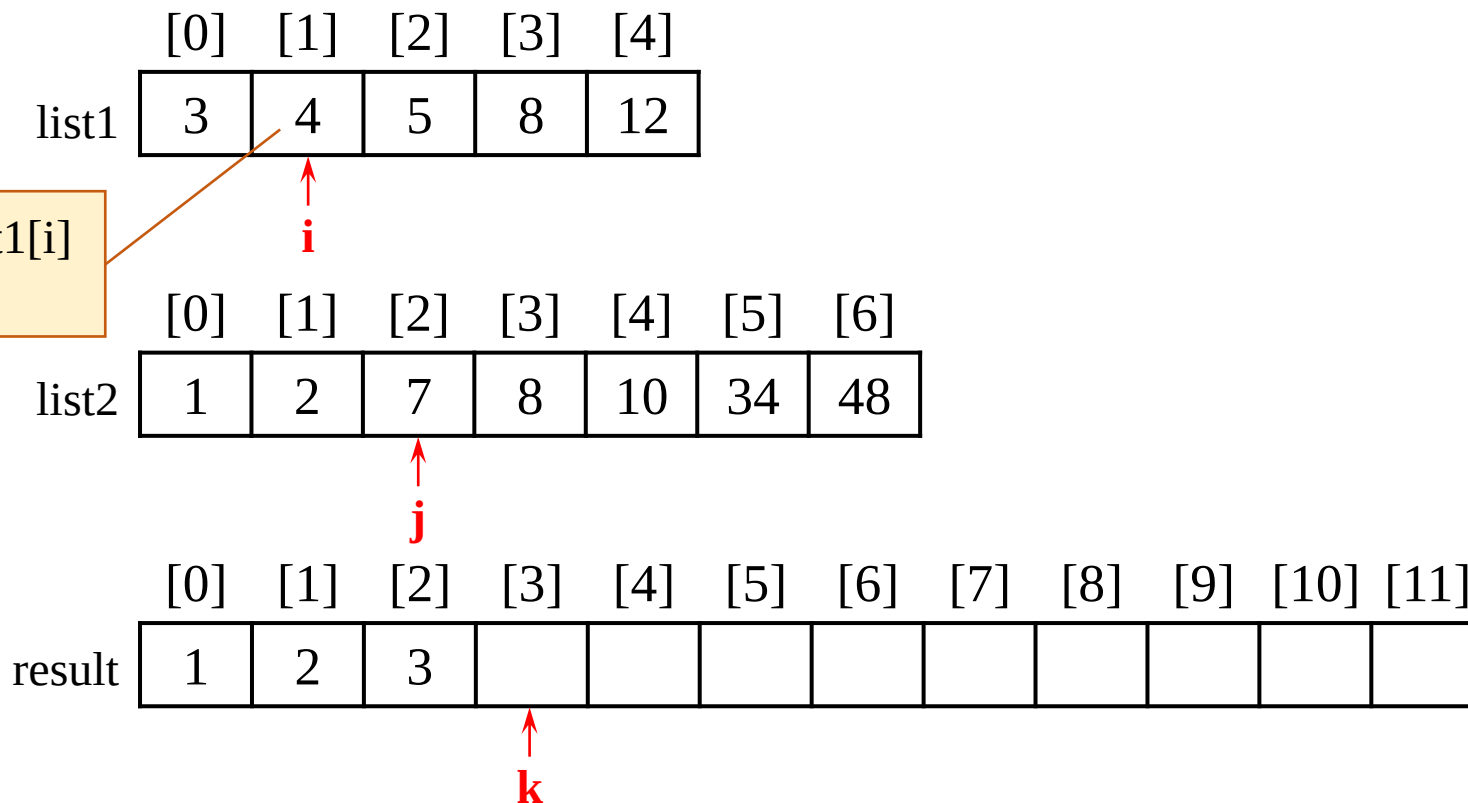


```

void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}

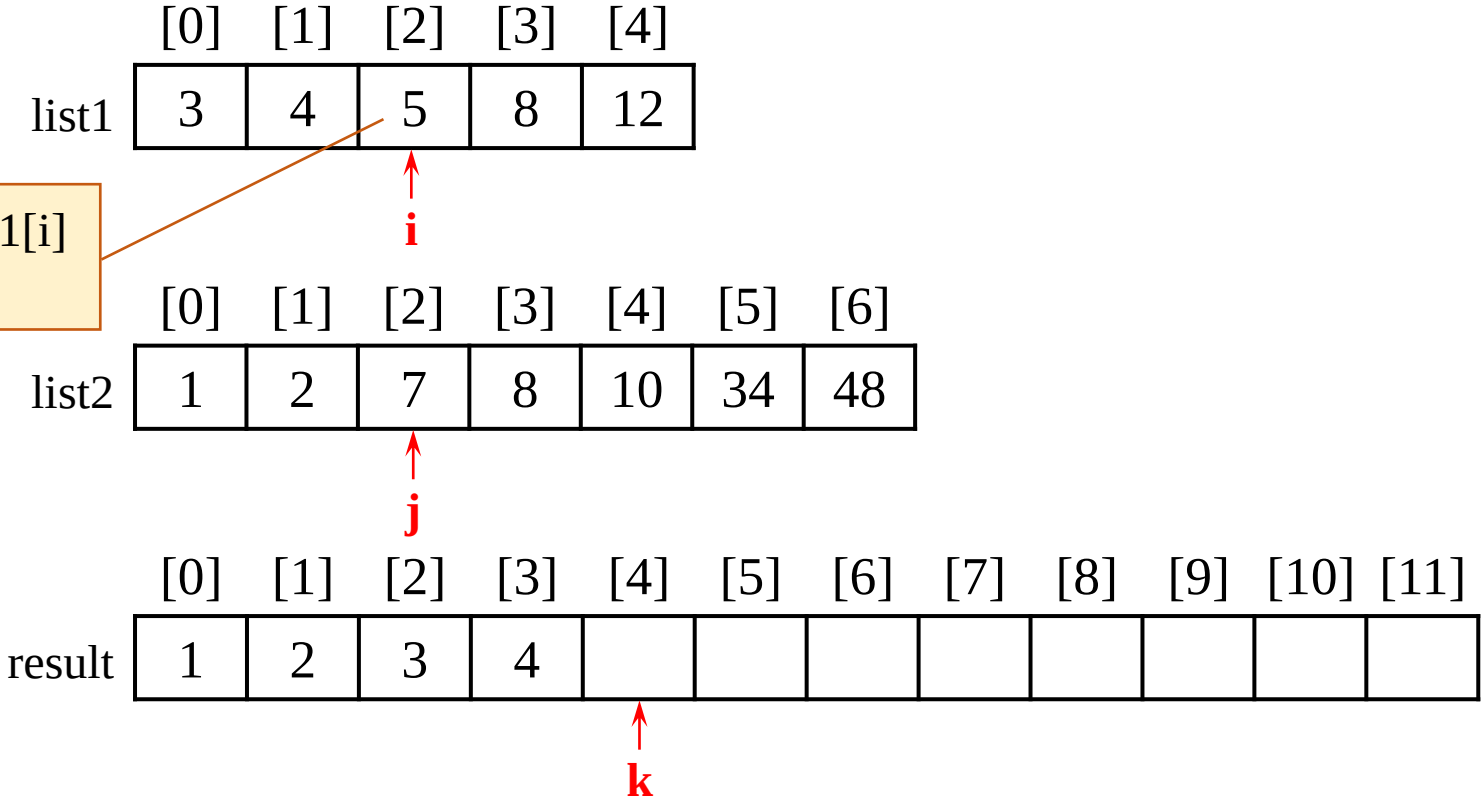
```

list1[i] <= list2[j]为真，提取list1[i]
赋给result[k]，k增1，i增1

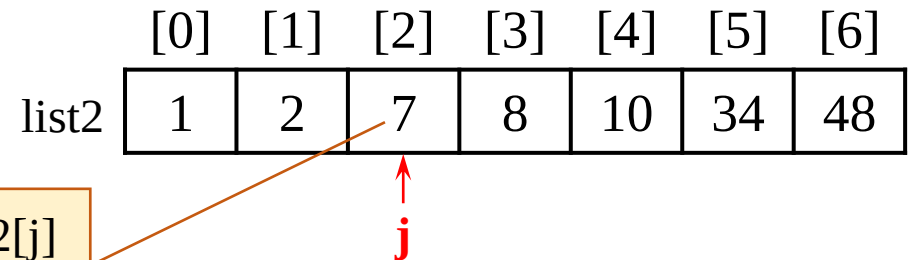
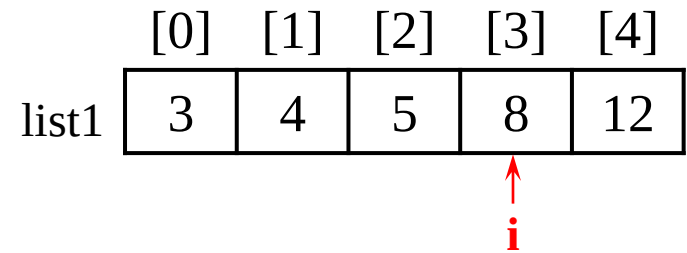


```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

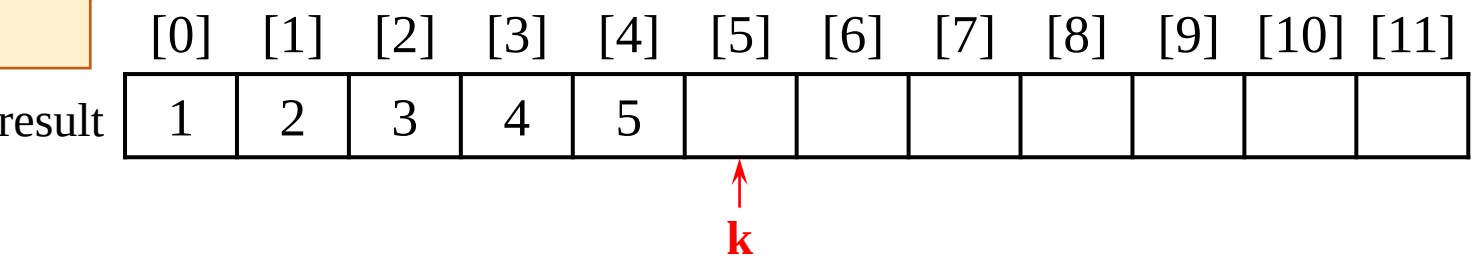
list1[i] <= list2[j]为真，提取list1[i]
赋给result[k]，k增1，i增1



```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

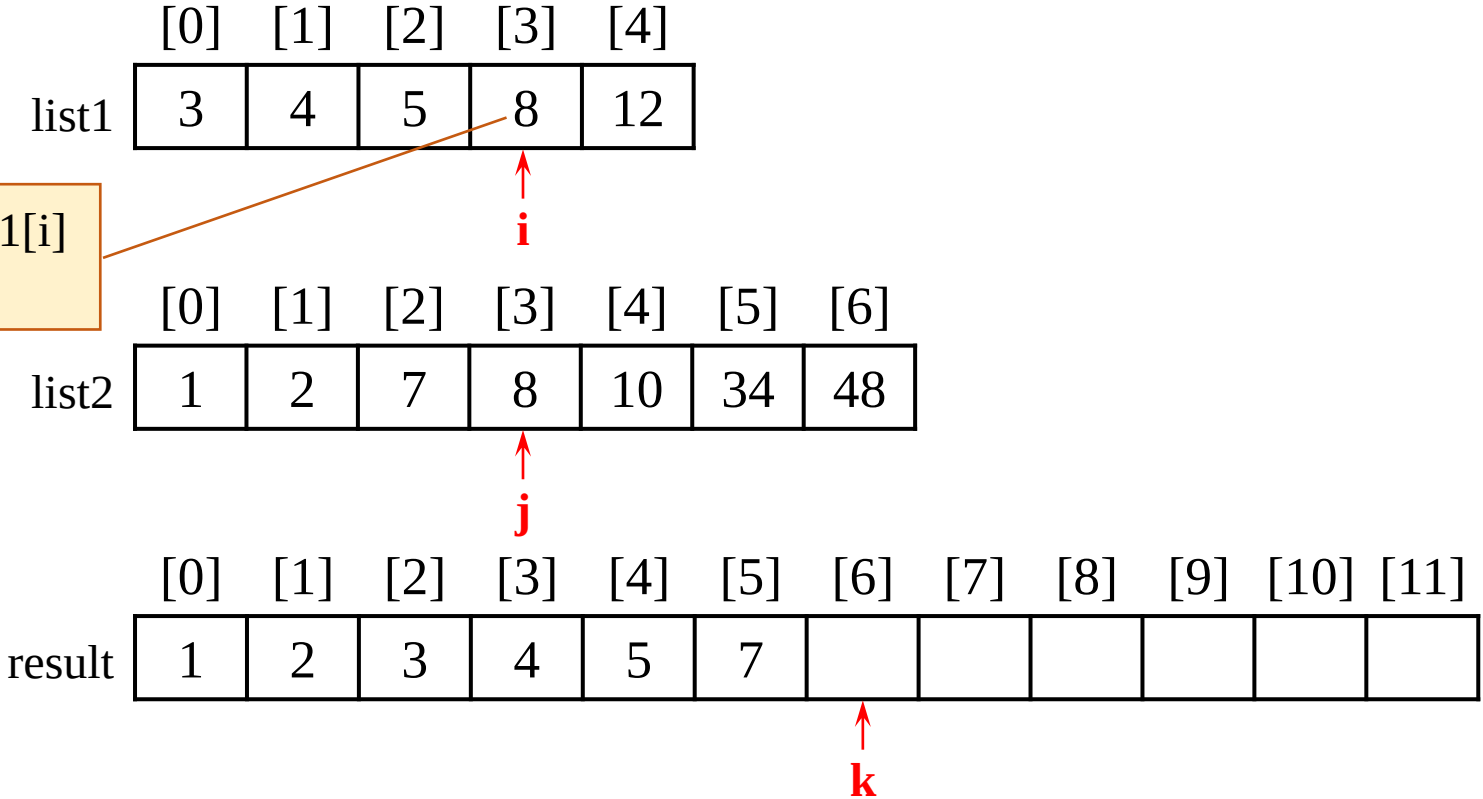


list1[i] <= list2[j]为假，提取list2[j]
赋给result[k], k增1, j增1

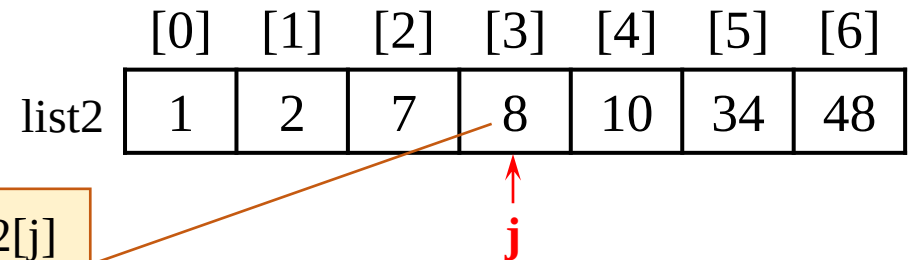
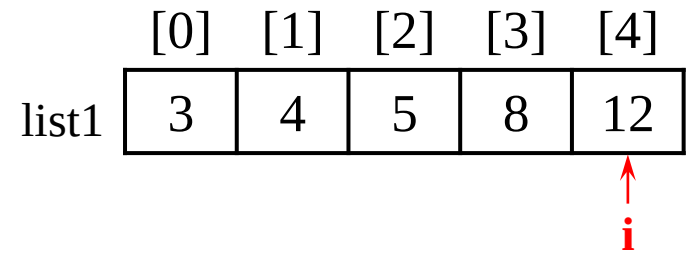


```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

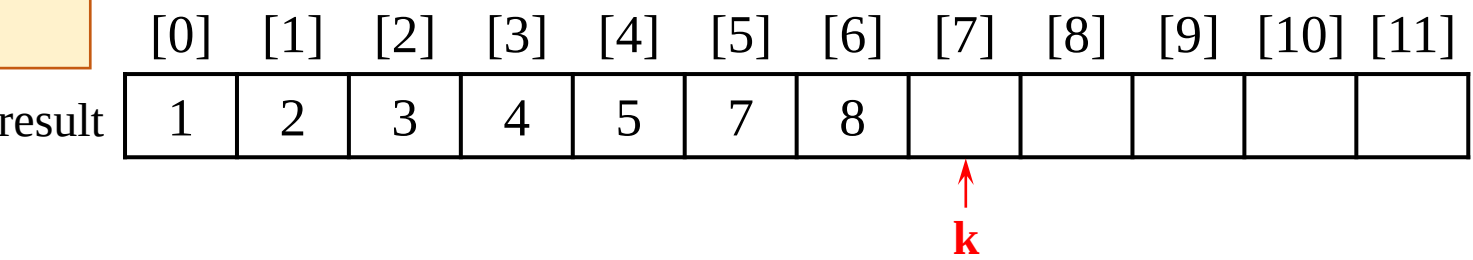
list1[i] <= list2[j]为真，提取list1[i]
赋给result[k]，k增1，i增1



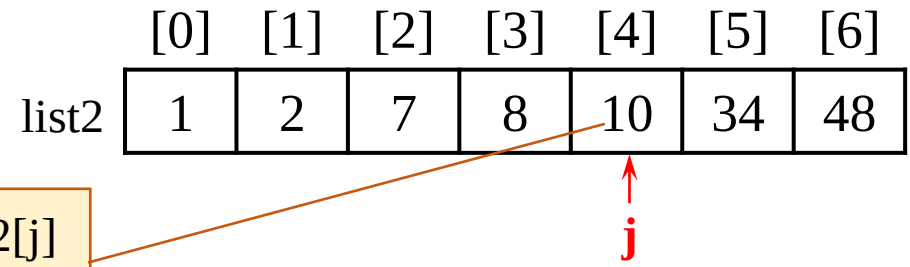
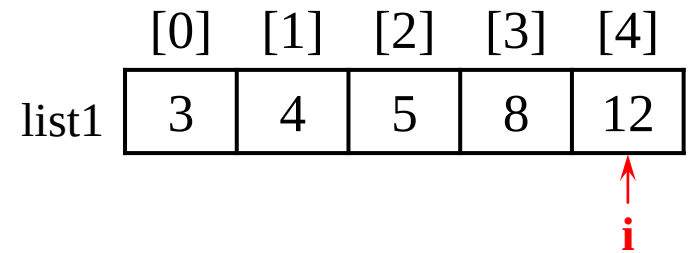
```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```



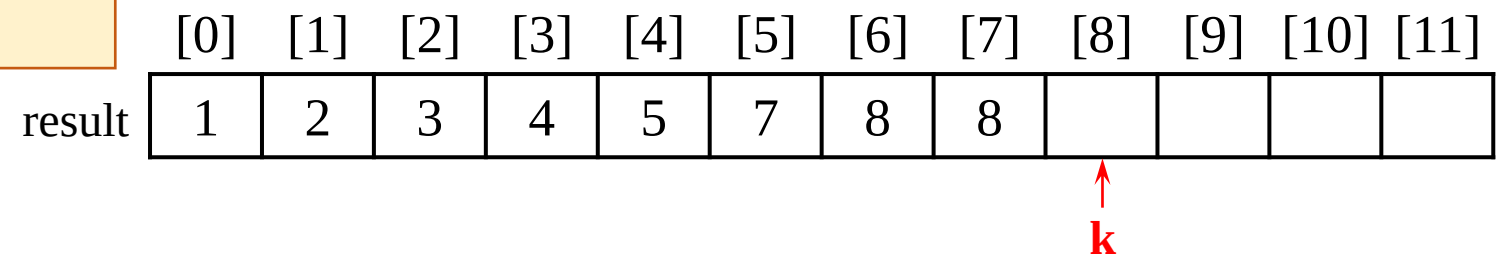
list1[i] <= list2[j]为假，提取list2[j]
赋给result[k]，k增1，j增1




```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

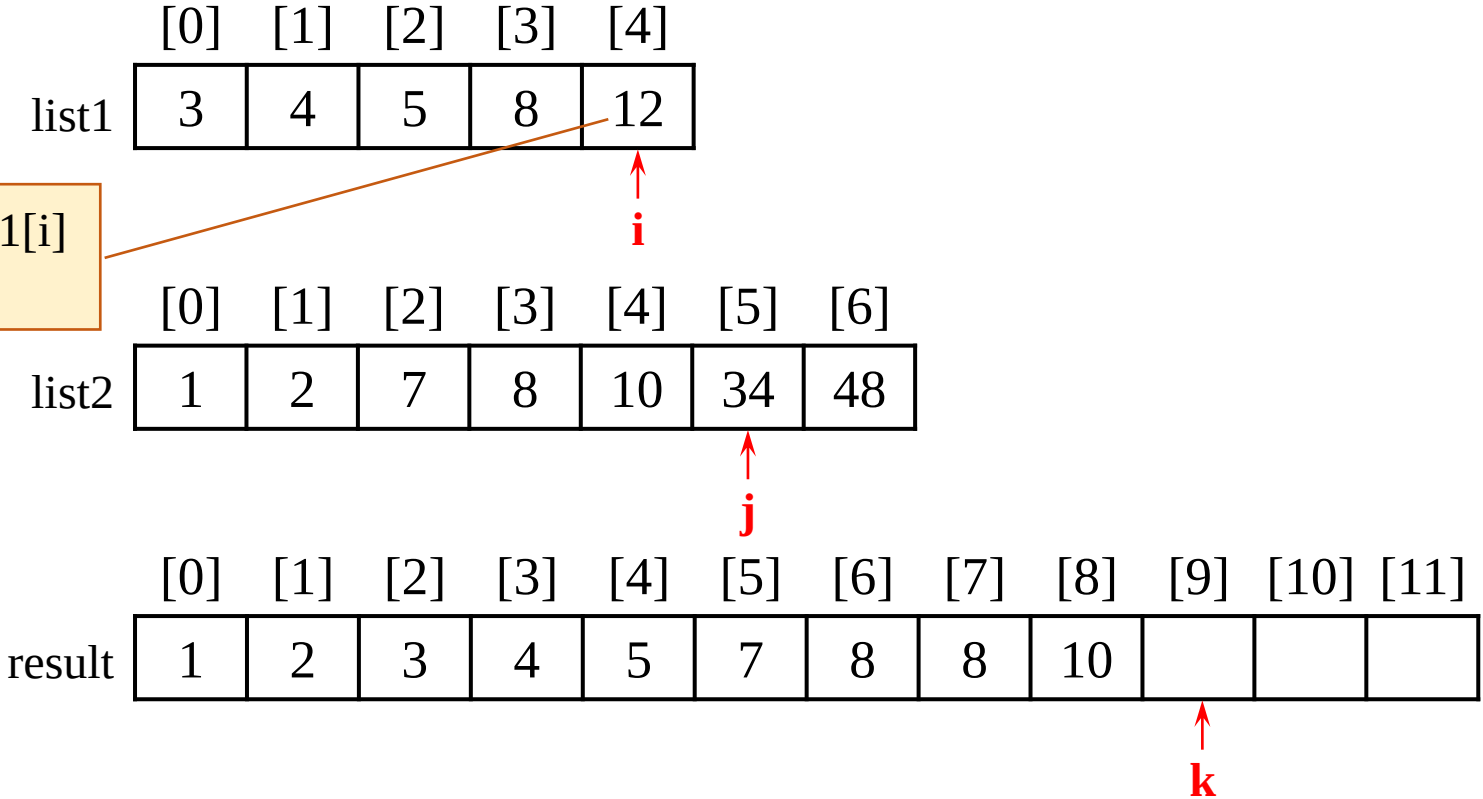


list1[i] <= list2[j]为假，提取list2[j]
赋给result[k]，k增1，j增1



```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

list1[i] <= list2[j]为真，提取list1[i]
赋给result[k]，k增1，i增1



```
void merge(int list1[], int list2[], int size1, int size2, int result[])
{
    int i=0, j=0, k=0;
    while(i < size1 && j < size2)
        result[k++] = (list1[i] <= list2[j]) ? list1[i++] : list2[j++];
    while(i < size1)
        result[k++] = list1[i++];
    while(j < size2)
        result[k++] = list2[j++];
}
```

i < size1为假

list1

[0]	[1]	[2]	[3]	[4]
3	4	5	8	12

↑
i

list2

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	2	7	8	10	34	48

↑
j

j < size2为真，复制list2[]中剩余数到result[]中

result

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]
1	2	3	4	5	7	8	8	10	12		

↑
k

本章主要内容



7.1 数组的定义与简单使用

7.2 数组作为函数参数

7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回

7.5 搜索数组



7.6 排序数组

7.7 C字符串

7.8 作业与实践

顺序搜索法



◆ 搜索数组指在数组中查找一个指定元素，例如，查找一个特定成绩是否出现在成绩列表中。常用搜索方法包括**顺序搜索**和**二分搜索**

■ 顺序搜索是将关键字key按顺序与数组中每个元素进行比较，直至关键字与某个数组元素匹配，或全部数组元素都已比较完毕，若未找到与关键字匹配的数组元素。**若发现匹配元素，则返回该元素的下标，否则，返回-1**

```
int linearSearch(const int list[ ], int key, int arraySize)
{
    for(int i = 0; i < arraySize; i++)
        if(key == list[i]) return i;
    return -1;
}
```

函数测试语句：

```
int list[ ] = {1, 4, 4, 2, 5, -3, 6, 2};
int i = linearSearch(list, 4, 8); // 返回值1
int j = linearSearch(list, -4, 8); // 返回值-1
int k = linearSearch(list, -3, 8); // 返回值5
cout << i << " , " << j << " , " << k << endl;
```

> D:\Edu-CPP\chap07\ex07-12.exe

1, -1, 5

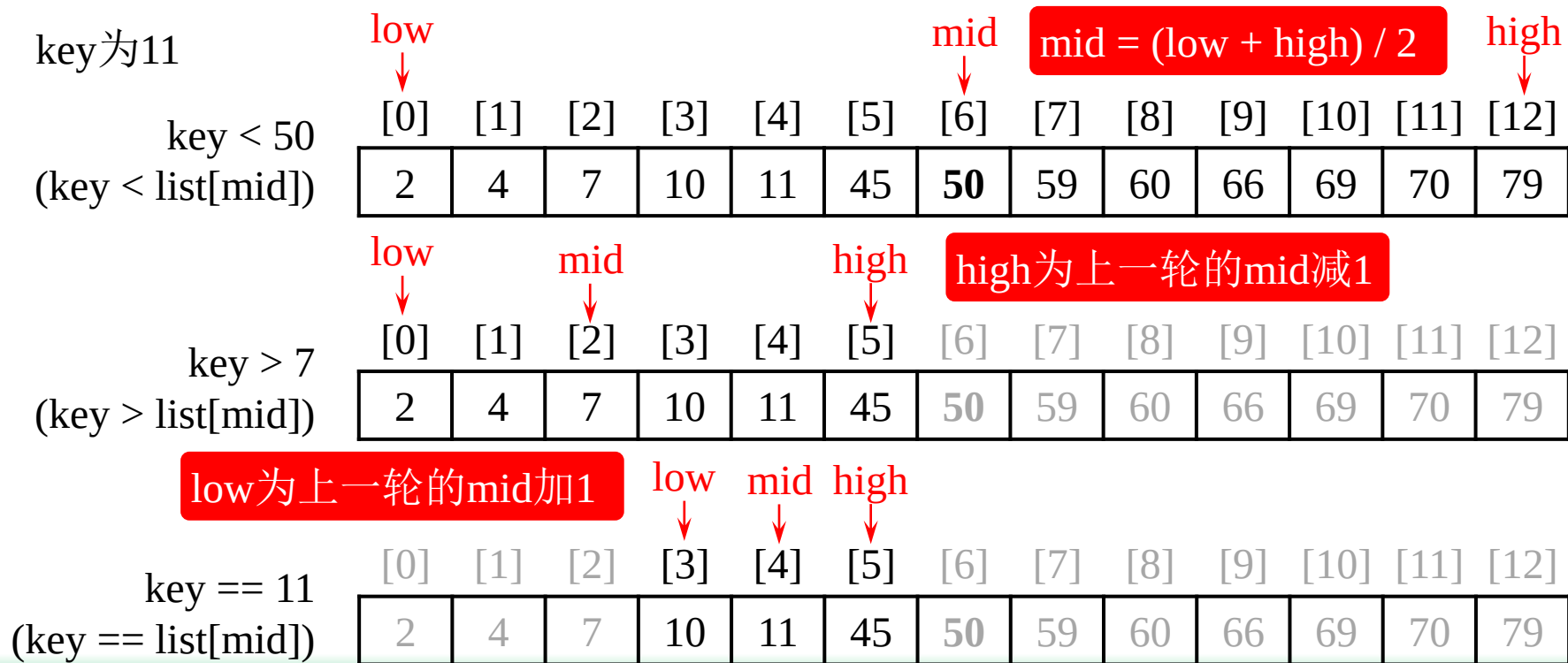
■ 若一个数组已排序（数组元素按升序或降序存储），则可以采用二分搜索法更高效地搜索



二分搜索法

◆ 假设数组元素升序存储，二分搜索法（也称作**折半搜索法**）首先将关键字与位于数组中间的元素进行比较，比较的结果有三种可能：

- 若关键字小于中间元素，只需继续在数组的前半部分进行搜索
- 若关键字与中间元素相等，则搜索结束，找到匹配的元素
- 若关键字大于中间元素，只需继续搜索数组的后半部分





◆ 二分搜索法的函数实现

```
#include <iostream>
using namespace std;

int binarySearch(const int list[], int key, int listSize);

int main()
{
    int list[] = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};
    int i = binarySearch(list, 2, 13); // returns 0
    int j = binarySearch(list, 11, 13); // returns 4
    int k = binarySearch(list, 12, 13); // returns -6
    int l = binarySearch(list, 1, 13); // returns -1
    int m = binarySearch(list, 3, 13); // returns -2

    cout << i << " " << j << " " << k << " " << l << " " << m << endl;
    return 0;
}
```

```
> D:\Edu-CPP\chap07\ex07-13.exe
```

```
0 4 -6 -1 -2
```

```
int binarySearch(const int list[], int key, int listSize)
{
    int low = 0;
    int high = listSize - 1;
    while (high >= low)
    {
        int mid = (low + high) / 2;
        if (key < list[mid])
            high = mid - 1;
        else if (key == list[mid])
            return mid;
        else
            low = mid + 1;
    }

    return -low - 1;
}
```

替换为**high > low**,
结果会如何?

本章主要内容



7.1 数组的定义与简单使用

7.2 数组作为函数参数

7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回

7.5 搜索数组

7.6 排序数组

7.7 C字符串

7.8 作业与实践



选择排序



◆每次从待排序的数组元素中选出最小（或最大）的元素，存放在已排好序列的起始位置（或末尾位置），直到全部待排序的数组元素排完

{49, 38, 65, 97, 76, **13**, 27, 49}

第1趟 {**13**, } {38, 65, 97, 76, 49, **27**, 49}

第2趟 {**13**, **27**, } {65, 97, 76, 49, **38**, 49}

第3趟 {**13**, **27**, **38**, } {97, 76, **49**, 65, 49}

第4趟 {**13**, **27**, **38**, **49**, } {76, 97, 65, **49**}

第5趟 {**13**, **27**, **38**, **49**, **49**, } {97, **65**, 76}

第6趟 {**13**, **27**, **38**, **49**, **49**, **65**, } {97, **76**}

第7趟 {**13**, **27**, **38**, **49**, **49**, **65**, **76**, } {**97**}

```
void selectSort(int a[ ], int arraySize)
{
    for(int loop = 0; loop < arraySize - 1; loop++)
    {
        int index = loop;
        for(int i = loop + 1; i < arraySize; i++)
            if(a[i] < a[index]) index = i;
        int temp = a[loop];
        a[loop] = a[index];
        a[index] = temp;
    }
}
```

```

#include <iostream>
using namespace std;

void selectSort(int [ ], int);
void print(int [ ], int);

int main()
{
    const int MAX_NUM = 8;
    int loop;
    int array[MAX_NUM] = {49, 38, 65, 97, 76, 13, 27, 49};

    print(array, MAX_NUM);
    selectSort(array, MAX_NUM);
    print(array, MAX_NUM);

    return 0;
}

```

 D:\Edu-CPP\chap07\ex07-14.exe

```

49 38 65 97 76 13 27 49
13 27 38 49 49 65 76 97

```

```

void selectSort(int a[ ], int arraySize)
{
    for(int loop = 0; loop < arraySize - 1; loop++)
    {
        int index = loop;
        for(int i = loop + 1; i < arraySize; i++)
            if(a[i] < a[index]) index = i;
        int temp = a[loop];
        a[loop] = a[index];
        a[index] = temp;
    }
}

void print(int a[], int arraySize)
{
    for(int loop=0; loop < arraySize; loop++)
        cout << a[loop]<<" ";
    cout<<"\n";
}

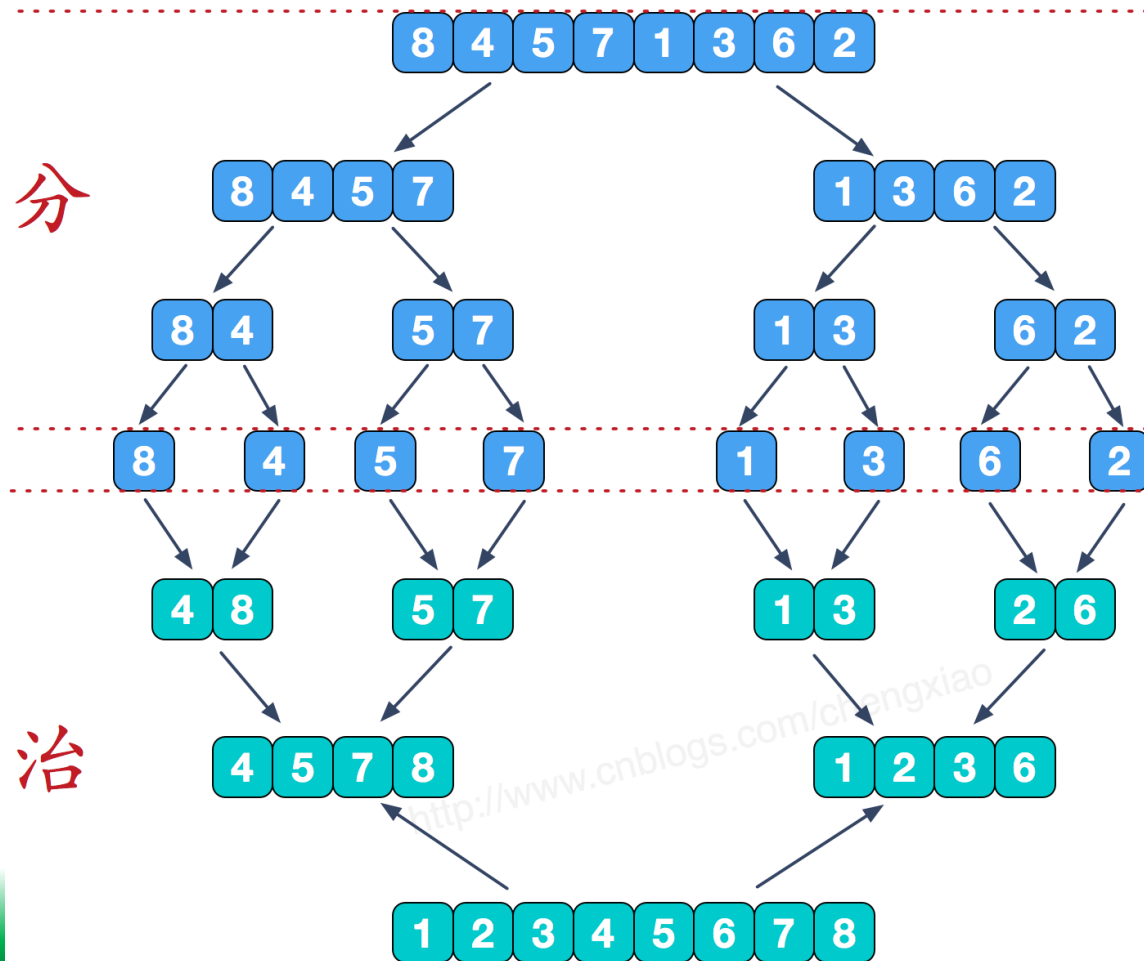
```



归并排序

◆ 采用经典的分治策略，即分而治之

- **分的阶段**将问题分成规模小的问题，**递归**求解
- **治的阶段**将分的阶段得到的各答案“归并”在一起



```

#include <iostream>
using namespace std;

void merge(int a[], int lmin, int rmin, int rmax) {
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

等价于：
a[lmin] = temp;
lmin++;

```

void mergeSort(int a[], int left, int right) {
    if(left == right)
        return;
    int q = (left + right)/2;
    mergeSort(a, left, q);
    mergeSort(a, q+1, right);
    merge(a, left, q+1, right);
}

```

```

int main() {
    int a[9] = {7, 3, 4, 15, 6, 2, 20, 18, 3};
    for(int i=0; i<9; i++)
        cout << a[i] << " ";
    cout << "\n";
    mergeSort(a, 0, 8);
    for(int i=0; i<9; i++)
        cout << a[i] << " ";
    return 0;
}

```

手工推演代码执行过程

D:\Edu-CPP\chap07\ex07-15.exe

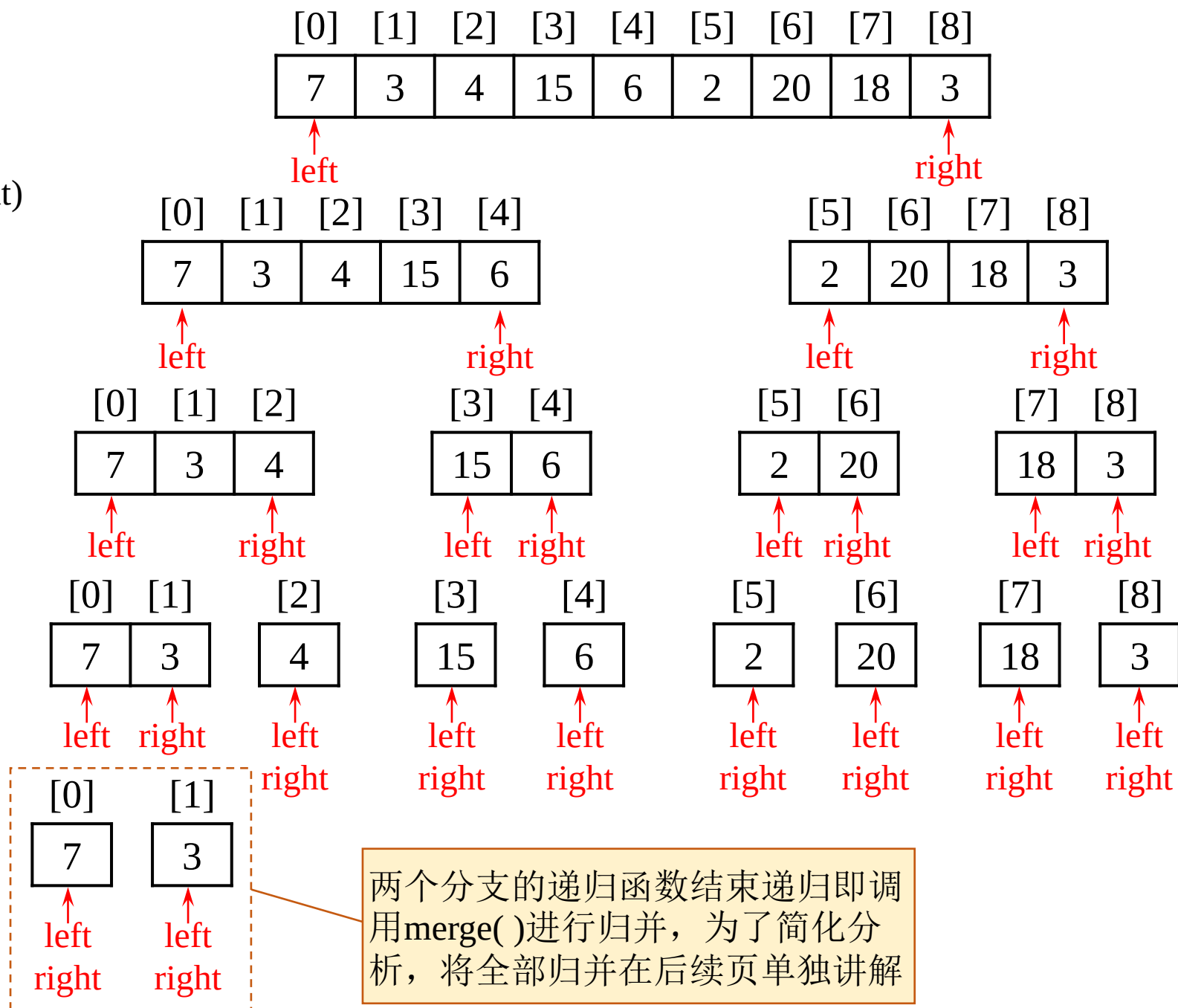
```

7 3 4 15 6 2 20 18 3
2 3 3 4 6 7 15 18 20

```

mergeSort(a, 0, 8);

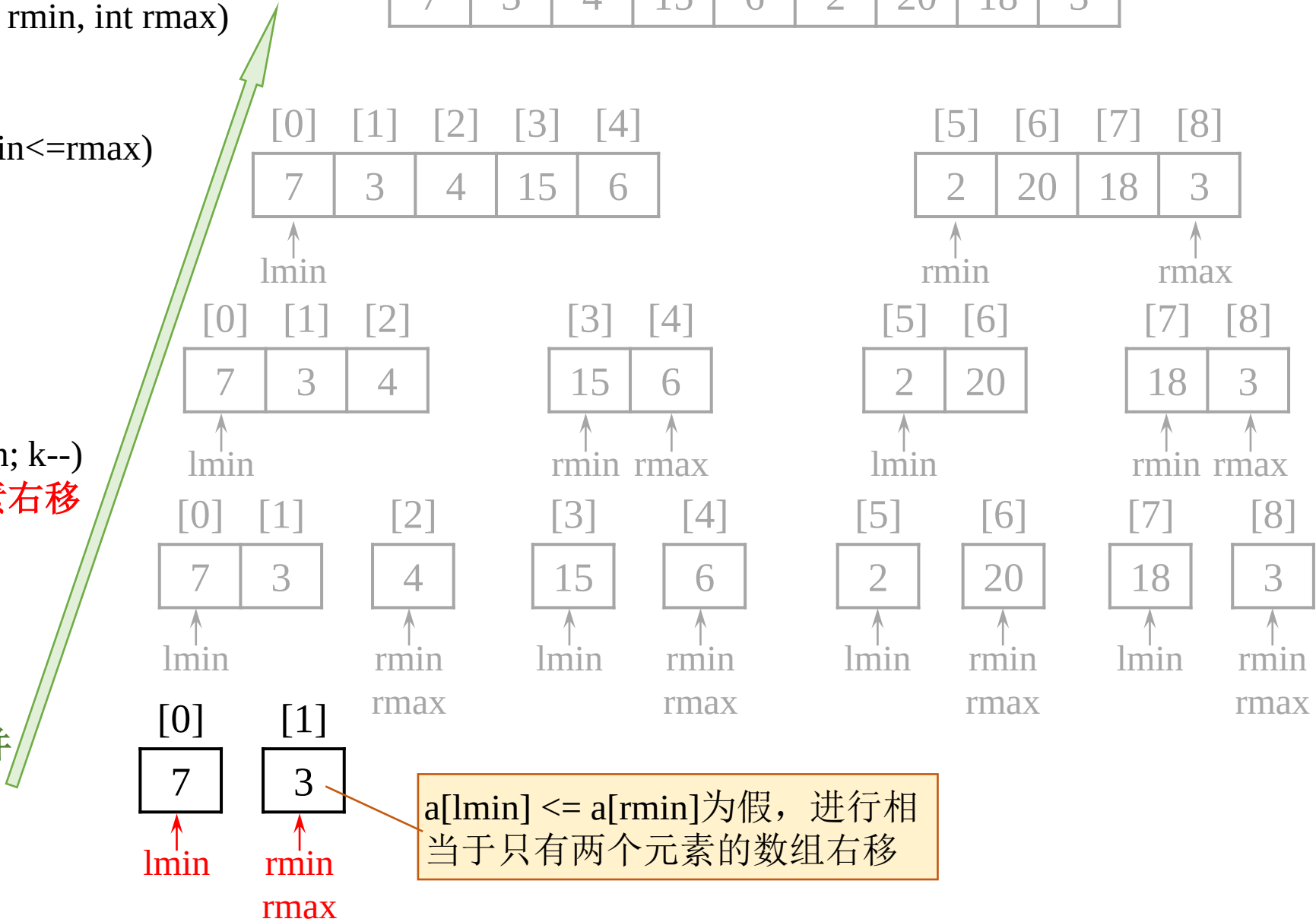
```
void mergeSort(int a[], int left, int right)
{
    if(left == right)
        return;
    int q = (left + right)/2;
    mergeSort(a, left, q);
    mergeSort(a, q+1, right);
    merge(a, left, q+1, right);
}
```



[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
7	3	4	15	6	2	20	18	3

```
void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}
```

归并



```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	7	4	15	6	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	7	4	15	6

[5]	[6]	[7]	[8]
2	20	18	3

[0]	[1]	[2]
3	7	4

[3]	[4]
15	6

[5]	[6]
2	20

[7]	[8]
18	3

[0]	[1]
3	7

[2]
4

[3]	[4]
15	6

[5]	[6]
2	20

[7]	[8]
18	3

[0]	[1]
3	7

↑
lmin
↑
rmin
↑
rmax

rmin <= rmax 为假, merge() 结束 while 循环并返回

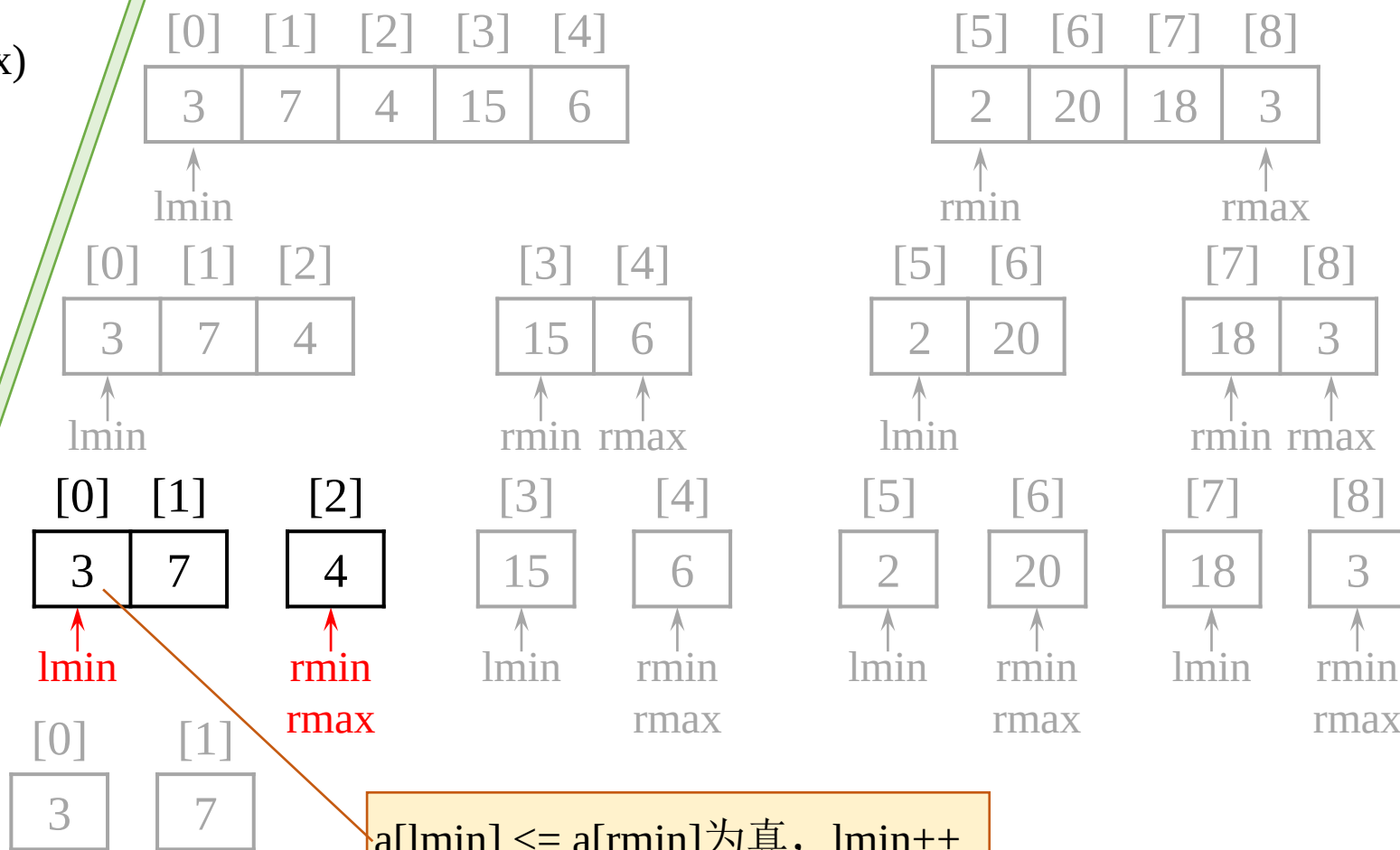
```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	7	4	15	6	2	20	18	3



a[lmin] <= a[rmin]为真, lmin++


```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	7	4	15	6	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	7	4	15	6

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	7	4

↑
lmin

[3]	[4]
15	6

↑ ↑
rmin rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑ ↑
rmin rmax

[0]	[1]	[2]
3	7	4

↑
lmin

↑
rmin

↑
rmax

[3]	[4]
15	6

↑
lmin

↑
rmin

↑
rmax

[5]	[6]
2	20

↑
lmin

↑
rmin

↑
rmax

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

a[lmin] <= a[rmin]为假，
进行数组元素右移

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	7	15	6	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	7	15	6

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	7

↑
lmin

[3]	[4]
15	6

↑ ↑
rmin rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑ ↑
rmin rmax

[0]	[1]
3	4

[2]
7

↑ ↑
lmin rmin
rmax

[3]	[4]
15	6

↑ ↑
lmin rmin
rmax

[5]	[6]
2	20

↑ ↑
lmin rmin
rmax

[7]	[8]
18	3

↑ ↑
lmin rmin
rmax

[0]	[1]
3	7

rmin <= rmax 为假，merge() 结束 while 循环并返回

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	7	15	6	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	7	15	6

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	7

↑
lmin

[3]	[4]
15	6

↑ ↑
rmin rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑ ↑
rmin rmax

[0]	[1]	[2]
3	4	7

[3]	[4]
15	6

↑
lmin

↑
rmin

↑
rmax

[5]	[6]
2	20

↑
lmin

↑
rmin

↑
rmax

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

[0]	[1]
3	7

$a[lmin] \leq a[rmin]$ 为假，进行相当于只有两个元素的数组右移

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	7	6	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	7	6	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	7

↑
lmin

[3]	[4]
6	15

↑ ↑
rmin rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑ ↑
rmin rmax

[0]	[1]	[2]
3	4	7

[3]	[4]
6	15

↑ ↑
lmin rmin
rmax

[5]	[6]
2	20

↑
lmin

[5]	[6]
2	20

↑ ↑
rmin rmax

[7]	[8]
18	3

↑
lmin

[7]	[8]
18	3

↑ ↑
rmin rmax

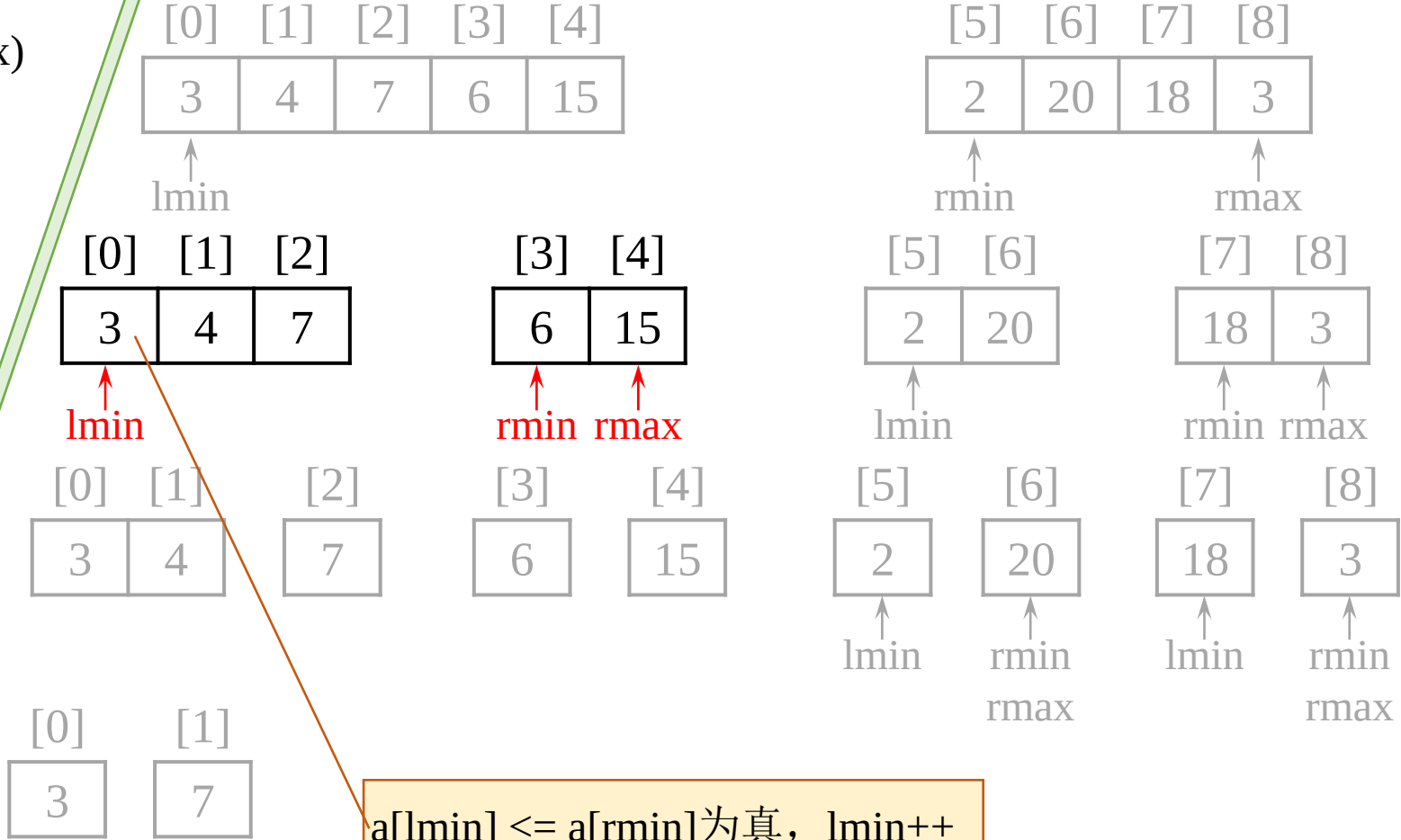
[0]	[1]
3	7

rmin <= rmax 为假，merge() 结束 while 循环并返回

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	7	6	15	2	20	18	3

```
void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}
```

归并



a[lmin] <= a[rmin]为真, lmin++

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	7	6	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	7	6	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	7

↑
lmin

[3]	[4]
6	15

↑ ↑
rmin rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑ ↑
rmin rmax

[0]	[1]	[2]
3	4	7

[3]	[4]
6	15

[5]	[6]
2	20

↑
lmin

↑
rmin

↑
rmax

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

[0]	[1]
3	7

$a[lmin] \leq a[rmin]$ 为真, $lmin++$

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	7	6	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	7	6	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	7

↑
lmin

[3]	[4]
6	15

↑
rmin rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑
rmin rmax

[0]	[1]	[2]
3	4	7

[3]	[4]
6	15

[5]	[6]
2	20

↑
lmin

↑
rmin
rmax

[7]	[8]
18	3

↑
lmin

↑
rmin
rmax

[0]	[1]
3	7

$a[lmin] \leq a[rmin]$ 为假，
进行数组元素右移

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

↑
lmin

↑
rmin
rmax

[5]	[6]
2	20

↑
lmin

[7]	[8]
18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	7

[3]	[4]
6	15

[5]	[6]
2	20

↑
lmin

↑
rmin

↑
rmax

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

[0]	[1]
3	7

a[lmin] <= a[rmin]为真, lmin++


```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	20

[7]	[8]
18	3

[0]	[1]	[2]
3	4	7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
18	3

[0]	[1]
3	7

lmin<=rmin-1为假，merge() 结束while循环并返回

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	20

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

↑
lmin

↑
rmin

↑
rmax

[0]	[1]
3	7

$a[lmin] \leq a[rmin]$ 为真, $lmin++$

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	18	3

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	20

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
18	3

↑
lmin
rmin
rmax

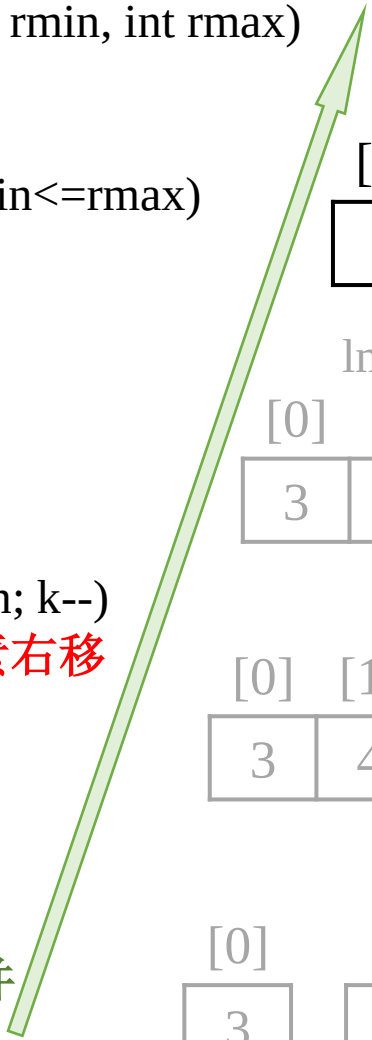
[0]	[1]
3	7

lmin <= rmin-1 为假，merge() 结束 while 循环并返回

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	18	3

```
void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}
```

归并



[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[0]	[1]
3	7

[5]	[6]	[7]	[8]
2	20	18	3

↑
rmin

↑
rmax

[5]	[6]
2	20

[7]	[8]
18	3

↑
lmin

↑
rmin

↑
rmax

[5]	[6]
2	20

[7]
18

[8]
3

↑
lmin

↑
rmin

↑
rmax

a[lmin] <= a[rmin]为假，进行相当于只有两个元素的数组右移

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	3	18

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	20	3	18

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	20

[7]	[8]
3	18

↑
lmin

↑
rmin rmax

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
3	18

↑
lmin rmin
↑
rmax

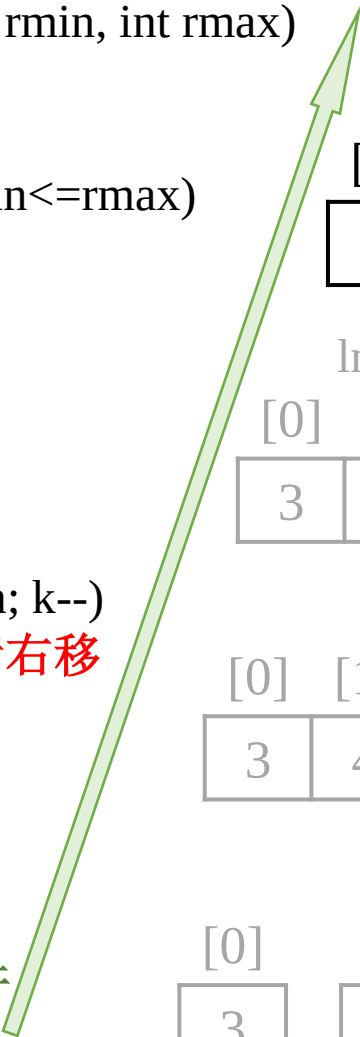
[0]	[1]
3	7

rmin<=rmax为假，merge()
结束while循环并返回

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	3	18

```
void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}
```

归并



[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[0]	[1]
3	7

[5]	[6]	[7]	[8]
2	20	3	18

↑
rmin

↑
rmax

[5]	[6]
2	20

[7]	[8]
3	18

↑
lmin

↑
rmin rmax

[5]	[6]
2	20

[7]	[8]
3	18

a[lmin] <= a[rmin]为真， lmin++

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	20	3	18

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[0]	[1]
3	4

[2]
7

[3]
6

[4]
15

[0]	[1]
3	7

[5]	[6]	[7]	[8]
2	20	3	18

↑
rmin

↑
rmax

[5]	[6]
2	20

[7]	[8]
3	18

↑
lmin

↑
rmin

↑
rmax

[5]
2

[6]
20

[7]
3

[8]
18

$a[lmin] \leq a[rmin]$ 为假，
进行数组元素右移

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	3	20	18

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[0]	[1]
3	4

[2]
7

[3]
6

[4]
15

[0]	[1]
3	7

[5]	[6]	[7]	[8]
2	3	20	18

↑
rmin

↑
rmax

[5]	[6]
2	3

[7]	[8]
20	18

↑
lmin

↑
rmin

↑
rmax

[5]
2

[6]
20

[7]
3

[8]
18

$a[lmin] \leq a[rmin]$ 为假，
进行数组元素右移


```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin<=rmin-1 && rmin<=rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	3	18	20

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	3	18	20

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	3

[7]	[8]
18	20

[0]	[1]	[2]
3	4	7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
3	18

↑ lmin
↑ rmin
↑ rmax

[0]	[1]
3	7

rmin<=rmax为假，merge()
结束while循环并返回

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	4	6	7	15	2	3	18	20

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

↑
lmin

[5]	[6]	[7]	[8]
2	3	18	20

↑
rmin

↑
rmax

[0]	[1]	[2]	[3]	[4]
3	4	6	7	15

[5]	[6]	[7]	[8]
2	3	18	20

[0]	[1]	[2]	[3]	[4]
3	4	7	6	15

[5]	[6]	[7]	[8]
2	20	3	18

[0]	[1]
3	7

a[lmin] <= a[rmin]为假，
进行数组元素右移

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
2	3	4	6	7	15	3	18	20

[0]	[1]	[2]	[3]	[4]
2	3	4	6	7

↑
lmin

[5]	[6]	[7]	[8]
15	3	18	20

↑
rmin

↑
rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	3

[7]	[8]
18	20

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
3	18

[0]	[1]
3	7

a[lmin] <= a[rmin] 为真, lmin++

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
2	3	4	6	7	15	3	18	20

[0]	[1]	[2]	[3]	[4]
2	3	4	6	7

lmin

[5]	[6]	[7]	[8]
15	3	18	20

rmin

rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	3

[7]	[8]
18	20

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
3	18

$a[lmin] \leq a[rmin]$ 为假，
进行数组元素右移

[0]	[1]
3	7

```

void merge(int a[], int lmin, int rmin, int rmax)
{
    int temp;
    while(lmin <= rmin-1 && rmin <= rmax)
    {
        if(a[lmin] <= a[rmin])
            lmin++;
        else
        {
            temp = a[rmin];
            for(int k=rmin; k>=lmin; k--)
                a[k] = a[k-1]; // 元素右移
            a[lmin++] = temp;
            rmin++;
        }
    }
}

```

归并

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
2	3	3	4	6	7	15	18	20

[0]	[1]	[2]	[3]	[4]
2	3	3	4	6

↑
lmin

[5]	[6]	[7]	[8]
7	15	18	20

↑ ↑
rmin rmax

[0]	[1]	[2]
3	4	6

[3]	[4]
7	15

[5]	[6]
2	3

[7]	[8]
18	20

[0]	[1]
3	4

[2]
7

[3]	[4]
6	15

[5]	[6]
2	20

[7]	[8]
3	18

[0]	[1]
3	7

本章主要内容



7.1 数组的定义与简单使用

7.2 数组作为函数参数

7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回

7.5 搜索数组

7.6 排序数组

7.7 C字符串

7.8 作业与实践





字符数组与C字符串

- ◆与定义整数数组、浮点数数组的方法一样定义字符数组

```
char city[ ] = {'S', 'h', 'a', 'n', 'g', 'h', 'a', 'i'};
```

- ◆若字符数组最后一个字符是空字符'\0'（也称作空终结符），则该字符数组称作**C风格字符串**（简称为**C字符串**或**字符串**）

```
char city1[ ] = {'S', 'h', 'a', 'n', 'g', 'h', 'a', 'i', '\0'};
```

```
char city2[ ] = "Shanghai"; // 在尾部自动添加'\0'
```

■city1[]和city2[]内容相同：

'S'	'h'	'a'	'n'	'g'	'h'	'a'	'i'	'\0'
-----	-----	-----	-----	-----	-----	-----	-----	------

city[0] city[1] city[2] city[3] city[4] city[5] city[6] city[7] city[8]

C字符串是包含空字符'\0'的特殊**字符数组**，并非任何字符数组都是C字符串

- 每个字符串值（例如"Shanghai"）都是一个C字符串，可以定义字符数组（例如city2[]）并用字符串值初始化该数组

C字符串在C语言中非常流行，但是被C++中更健壮、更方便、更有用的string类型代替了。本节介绍C字符串的目的是给出更多的数组使用实例，也可以解决C程序的一些遗留问题



◆ 定义C字符串时可以直接用字符串值赋值

```
char s[6] = { "Hello" };
```

```
char s[6] = "Hello" ;
```

```
char s[ ] = "Hello" ;
```

◆ 不能先定义再给它赋值

```
■ char s[10];
```

```
■ s[10]="china"; ❌
```

变更C字符串内容的方法，将在后面的C字符串函数等章节介绍

输入和输出C字符串



- ◆ 输出C字符串和输出普通变量一样，例如输出C字符串s的语句为：

```
cout << s;
```

- ◆ 从键盘读取一个C字符串，也与读取一个数类似：

```
char city[30];
```

```
cout << "Enter a city: ";
```

```
cin >> city;
```

```
cout << "Your entered " << city << endl;
```

```
D:\Edu-CPP\chap07\ex07-16.exe
Enter a city: HeFei
Your entered HeFei
```

```
D:\Edu-CPP\chap07\ex07-16.exe
Enter a city: He Fei
Your entered He
```

- city[]的长度为30，故输入不能超过29个字符（自动在字符串值末尾补'\0'）
- 不能读取包含空格的字符串，若需读取包含空格的字符串，则使用C++提供的cin.getline()函数

```
char city[30];
```

```
cout << "Enter a city: ";
```

```
cin.getline(city, 29, '\n');
```

```
cout << "Your entered " << city << endl;
```

```
D:\Edu-CPP\chap07\ex07-16.exe
Enter a city: He Fei
Your entered He Fei
```

C字符串函数



- ◆ C字符串以空字符'\0'结束，C++可以利用这个特点更有效地操作C字符串。当将C字符串传递给一个函数时，不需像传递整数数组那样同时传递其长度，因C字符串的长度可以根据C字符串自身获取：

```
unsigned int strlen(char s[ ])
{
    unsigned int i = 0;
    for( ; s[i] != '\0'; i++) ;
    return i;
}
```

- ◆ C++库函数中有很多与strlen()类似的函数，用于操作C字符串



◆ C字符串函数如表所示, 其中**atoi**、**atof**、**atol**定义在**cstdlib**头文件中, 其它在**cstring**头文件中, **size_t**是一个C++类型, 等同于**unsigned int**

函数	描述
<code>size_t strlen(char s[])</code>	返回字符串的长度, 即空字符之前的字符个数
<code>strcpy(char s1[], const char s2[])</code>	将字符串s2复制到s1中
<code>strncpy(char s1[], const char s2[], size_t n)</code>	将字符串s2中前n个字符复制到s1中
<code>strcat(char s1, const char s2[])</code>	将字符串s2拼接到s1尾部
<code>strncat(char s1[], const char s2[], size_t n)</code>	将字符串s2中前n个字符拼接到s1尾部
<code>int strcmp(const char s1[], const char s2[])</code>	通过字符的ASCII码对比, 返回大于0、等于0或小于0的数, 代表s1大于、等于或小于s2
<code>int strcmp(const char s1[], const char s2[], size_t n)</code>	类似strcmp(), 但指定比较s1和s2中的前n个字符
<code>int atoi(char s[])</code>	返回字符串对应的int型值
<code>double atof(char s[])</code>	返回字符串对应的double型值
<code>long atoll(char s[])</code>	返回字符串对应的long型值
<code>void itoa(int value, char s[], int radix)</code>	将整数value转换为字符串s, radix指定按什么进制转换

使用strcpy和strncpy函数复制字符串



◆复制C字符串的常见错误是：

```
char city[30] = "Shanghai";
```

```
city = "Beijing"; ❌
```

◆实现字符串的复制，可以使用：

```
strcpy(city, "Beijing");
```

◆strncpy()类似strcpy()，但可以指定需复制的字符数n：

```
char city[30];
```

```
strncpy(city, "Beijing", 3); // 将前3个字符"Bei"复制给city
```

- 若n小于或等于源字符串的长度，空字符不会复制到目标字符串city中；若n大于源字符串的长度，则源字符串的全部内容，包括尾部的空字符，被复制到目标字符串city

◆strcpy()和strncpy()存在目标字符串（数组）长度不足而越界的可能（缓冲区溢出），应检查数组的界限以确保安全复制

使用strcat和strncat函数拼接字符串



- ◆ **strcat()**用于将第2个参数的字符串拼接 to 第1个参数字符串的尾部，第1个参数在调用函数前须分配足够内存

```
char s1[7] = "abc";  
char s2[4] = "def";  
strcat(s1, s2);  
cout << s1 << endl; // 输出: abcdef
```

- ◆ **strncat()**与**strcat()**类似，但可以用第3个参数指定需拼接 to 第1个参数字符串尾部的字符个数

```
char s[9] = "abc";  
strncat(s, "ABCDEF", 3);  
cout << s << endl; // 输出: abcABC
```

- ◆ **strncat()**与**strcat()**都有可能使第一个参数数组越界（缓冲区溢出），应检查数组的界限以确保安全拼接

使用strcmp和strncmp函数比较字符串



- ◆ **strcmp()**通过逐一比较第1个参数与第2个参赛字符串中每个字符的ASCII码值大小，返回值大于0、等于0或小于0，代表第1个参数字符串大于、等于或小于第2个参赛字符串

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
int main()
{
    char s1[ ] = "Good morning";
    char s2[ ] = "Good afternoon";
```

```
int result = strcmp(s1, s2);
if(result > 0)
    cout << "s1 is greater than s2" << endl;
else if(result == 0)
    cout << "s1 is equal to s2" << endl;
else
    cout << "s1 is less than s2" << endl;

cout << "strncmp(s1, s2, 4): " << strncmp(s1, s2, 4) << endl;
return 0;
}
```

D:\Edu-CPP\chap07\ex07-17.exe

```
s1 is greater than s2
strncmp(s1, s2, 4): 0
```

字符串和数值之间的转换



- ◆ 利用 `atoi`、`atol` 等函数将字符串转换为相应类型的数值，利用 `itoa()` 将整数按指定的进制转换为字符串

部分编译器不支持函数 `itoa()`

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main() {
    char s1[ ] = "65", s2[ ] = "4";
    cout << "atoi: " << atoi(s1) + atoi(s2) << endl;
    char s3[ ] = "65.5", s4[ ] = "4.4";
    cout << "atof: " << atof(s3) + atof(s4) << endl;
    char s5[15], s6[15], s7[15];
    itoa(100, s5, 2); // 按二进制转换为字符串
    itoa(100, s6, 8); // 按八进制转换为字符串
    itoa(100, s7, 16); // 按十六进制转换为字符串
    cout << 100 << "的二进制是" << s5 << ", 八进制是" << s6 << ", 十六进制是" << s7 << endl;
    return 0;
}
```

将"65"分别改为"65.9"、"65a"，结果不变，
即 `atoi()` 仅转换非数字字符之前的数字字符

D:\Edu-CPP\chap07\ex07-18.exe

```
atoi: 69
atof: 69.9
100的二进制是1100100，八进制是144，十六进制是64
```

小结



◆ C++有两种类型的字符串表示形式

- C-风格字符串，简称C字符串（字符数组，'\0'作为结束标志）
- C++引入的string类类型

◆ 对字符串操作两种方式

- C字符串函数：<cstring>是C标准库头文件
- 操作string字符串的库函数：<string>是C++标准库头文件

本章主要内容



7.1 数组的定义与简单使用

7.2 数组作为函数参数

7.3 防止函数修改传递参数的数组

7.4 数组作为函数值返回

7.5 搜索数组

7.6 排序数组

7.7 C字符串

7.8 作业与实践



第7章作业



(1)编写程序读入至多100个1~100之间的整数，输出每个数出现的次数。假定输入0时表示输入结束。一个运行示例是：

Enter the integers between 1 and 100: 2 5 43 2 0

2 occurs 2 times

5 occurs 1 time

43 occurs 1 time

■注意：若一个数字出现超过一次，则要使用复数形式的times

(2)编写程序读入若干成绩，数目未定，分析有多少成绩在平均成绩之上、之下及恰好相等。输入负数表示输入结束。假定成绩最高为100分

(3)编写程序读入10个数，输出其中不同的数（即若一个数出现多次，只输出一一次）。（提示：读入的数若是一个新的值，则将其存入一个数组，否则将其丢弃。输入完毕后，数组中保存的就是不同的数）



(4)编写一个函数**`int indexOfSmallestElement(int array[ ], int size)`**, 返回一个整型数组中最小元素的下标, 若有多个最小元素, 返回最小下标

(5)如果两个数组**`list1`**和**`list2`**具有相同的长度, 且对于每个**`i`**都有**`list1[i]`**和**`list2[i]`**相同, 则称它们为严格相同 (**`strictly identical`**)。编写函数**`bool strictlyEqual(const int list1[ ], const int list2[ ], int size)`**, 编写测试程序提示输入两个整数数组, 并显示它们是否严格相同。样例运行如下, 注意输入数据中的第一个数是数组元素个数, 不是数组的元素。假定数组大小不超过20。

```
Enter list1: 5 2 5 6 1 6
Enter list2: 5 2 5 6 1 6
Two lists are strictly identical
```

```
Enter list1: 5 2 5 6 6 1
Enter list2: 5 2 5 6 1 6
Two lists are not strictly identical
```



(6)编写函数`bool isConsecutiveFour(const int values[ ], int size)`检查数组是否含有4个连续元素具有相同值。编写测试程序提示输入一系列整数，并显示它们是否含有4个连续相同数。程序应首先提示输入整数数量，即数组元素个数，假定最大数量是80。运行样例如下：

```
Enter the number of values: 8
Enter the values: 3 4 5 5 5 5 4 5
The list has consecutive fours
```

```
Enter the number of values: 9
Enter the values: 3 4 5 5 6 5 5 4 5
The list has no consecutive fours
```

(7)编写函数`bool isSorted(const int list[ ], int size)`，若数组已升序排序，则返回`true`，否则返回`false`。编写测试程序提示输入一个数组并显示其是否是已排序的。运行样例如下，注意输入的第一个数是数组元素个数，不是数组元素。假定数组大小不超过80。

```
Enter list: 8 10 1 5 16 61 9 11 1
The list is not sorted
```

```
Enter list: 10 1 1 3 4 4 5 7 9 11 21
The list is sorted
```



(8)编写程序提示用户输入两个含有10个元素的整数数组，并显示同时出现在两个数组中的共有元素。运行样例如下：

```
Enter list1: 8 5 10 1 6 16 61 9 11 2
Enter list2: 4 2 3 10 3 34 35 67 3 1
The common elements are 10 1 2
```

(9)编写函数**int countLetters(const char s[])**，返回C字符串中字母数量。编写测试程序读入一个C字符串，并显示其中字母数量。运行样例如下：

```
Enter a string: 2025 is coming
The number of letters in "2025 is coming" is 8
```

(10)编写函数**void ftoa(double f, char s[])**将浮点数转换为C字符串。编写测试程序提示输入一个浮点数，显示每个数字和小数点，并且用空格隔开。运行样例如下：

```
Enter a number: 232.45
The number is 2 3 2 . 4 5
```